

Projektarbeit

LEGO::Lab: Schnittstellen zu externen Displays

im Studiengang
Informatik Master

vorgelegt von

Florian Kaiser
Matrikelnummer: 79728
kaf11018@h-ka.de

an der Hochschule Karlsruhe
am 17.09.2026

Prüfer: Uwe Haneke, Matthias Mruzek-Vering

ERKLÄRUNG

Hiermit erkläre ich, dass ich die vorliegende Arbeit eigenständig und ohne fremde Hilfe angefertigt habe. Textpassagen, die wörtlich oder dem Sinn nach auf Publikationen oder Vorträgen anderer Autoren beruhen, sind als solche kenntlich gemacht. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Karlsruhe, 17.09.2026

Florian Kaiser

ERKLÄRUNG ZUR NUTZUNG VON KI-WERKZEUGEN

Bei der Erstellung dieser Arbeit wurden KI-Werkzeuge unterstützend eingesetzt, insbesondere für Teile des Programmcodes sowie für Textentwürfe. Alle inhaltlichen und konzeptionellen Entscheidungen, die Durchführung und Auswertung, die Überprüfung sämtlicher Ergebnisse sowie die endgültige Fassung des Textes wurden von den Autor:innen selbst erstellt und geprüft.

Bei der Nutzung wurden die allgemein anerkannten Standards guter wissenschaftlicher Praxis sowie die einschlägigen Vorgaben des Fachbereichs zum Umgang mit KI-generierten Inhalten beachtet. Jede wesentliche Behauptung wurde eigenständig auf Plausibilität und Richtigkeit geprüft und durch geeignete Quellen belegt. Es wurden keine personenbezogenen, urheberrechtlich geschützten oder vertraulichen Daten an die KI-Systeme übermittelt. Die eigenständige Leistung der Autor:innen ist durchgängig gewahrt.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung	1
1.2	Zielsetzung der Arbeit	1
1.3	Aufbau der Arbeit	2
2	Grundlagen und verwandte Arbeiten	2
2.1	Entfernter Prozeduraufruf und Middleware	3
2.2	Serialisierung von Nachrichten	3
2.3	Eingebettete Systeme und ihre Kommunikation	4
2.4	Einordnung und Auswahlkriterien	5
2.5	Etablierte Rahmenwerke für entfernte Aufrufe	5
2.6	Serialisierung für ressourcenarme Geräte	6
2.7	Protokolle für eingebettete Systeme und Sensornetze	6
2.8	Der betrachtete Bus und PUPRemote als Transportbasis	7
2.9	Abgrenzung	7
3	Systemanalyse und Anforderungen	8
3.1	Zielsystem und Hardwareumgebung	9
3.1.1	Der LEGO Education SPIKE Prime Hub als Ausgangsplattform	9
3.1.2	Notwendigkeit einer externen Erweiterung	9
3.2	Anforderungen an die Display-Integration	10
3.2.1	Anforderungen an ein externes Display	10
3.2.2	Anforderungen an einen externen Mikrocontroller	10
3.2.3	Resultierende Systemstruktur	10
3.3	Kommunikationsanforderungen	11
3.4	Einschränkungen der Plattform	11
3.4.1	Eingeschränkte Kommunikationsrichtung	12
3.4.2	Begrenzte Paketgröße	12
3.4.3	Beschränkte Laufzeitumgebung	12
3.4.4	Wechselwirkung der Einschränkungen	13
3.5	Verallgemeinerung des Problemraums	13
4	Architektur des Systems	14
4.1	Zielsetzung der Architektur	15
4.2	Schichtenmodell	15
4.2.1	Busanbindung	16
4.2.2	Transportschicht	16
4.2.3	Kommunikationsschicht	16
4.2.4	Objektmodell	17
4.2.5	Anwendung und Anzeige	17

4.3	Rollen im Entwurf	17
4.3.1	Interceptor	17
4.3.2	Dispatcher	17
4.3.3	Stellvertreter und Adapter	18
4.4	Kommunikationsmodell	18
4.5	Datenfluss im System	18
5	Entwurf von Transport- und Kommunikationsschicht	18
5.1	Motivation und Randbedingungen	19
5.2	Transportschicht	19
5.2.1	Interceptor-Abstraktion	19
5.2.2	Frame-Struktur	21
5.2.3	Nutzlastlänge und Formatunabhängigkeit	21
5.2.4	Kosten der Rahmung	22
5.2.5	Fragmentierung und Rekonstruktion	23
5.2.6	Serialisierung	24
5.2.7	Zuverlässigkeitsmechanismen	24
5.3	Kommunikationsschicht	25
5.3.1	Aufbau einer Aufrufnachricht	25
5.3.2	Zuordnung von Anfrage und Antwort	26
5.3.3	Verteilung eingehender Aufrufe	26
5.3.4	Anfragegetriebener Rückkanal	27
6	Referenzimplementierung	27
6.1	Laufzeitumgebung	28
6.2	Hardwareaufbau	28
6.2.1	Aufbau und Komponenten	28
6.2.2	Signalzuordnung und Aufbau der Adapterplatine	28
6.3	Busanbindung	28
6.4	Transportschicht	29
6.4.1	Frame	29
6.4.2	Message	30
6.4.3	Serialisierung	32
6.5	Kommunikationsschicht	33
6.5.1	Aufrufnachricht	33
6.5.2	Verteilung eingehender Aufrufe	34
6.5.3	Rückkanal	35
6.6	Objektmodell	36
6.6.1	Referenzen	36
6.6.2	Lebenszyklus	37
6.6.3	Veraltete Referenzen	37
6.7	Ansteuerung der Anzeige	38

6.8	Ablauf eines vollständigen Aufrufs	39
6.9	Nebenläufigkeit und Fehlerbehandlung	39
7	Evaluation	39
7.1	Funktionaler Nachweis	40
7.2	Übertragungsaufwand	40
7.3	Ressourcenverbrauch	40
7.4	Bewertung des Hardwareaufbaus	40
7.5	Diskussion der Ergebnisse	40
8	Fazit und Ausblick	40
8.1	Limitierungen	41
8.1.1	Kommunikationsmodell	41
8.1.2	Latenz und Durchsatz	41
8.1.3	Abstraktionsgrenzen	41
8.2	Übertragbarkeit auf andere Verbindungsklassen	41
8.3	Mögliche Erweiterungen	41
8.3.1	Ereignisgetriebener Rückkanal	41
8.3.2	Alternative Serialisierung	41
8.3.3	Integrierter Hardwareaufbau	41
8.3.4	Weitere Peripherieklassen	41
8.4	Zusammenfassung	41
9	Abbildungs- und Tabellenverzeichnis	42
10	Abkürzungsverzeichnis	43
	Bibliografie	44

1 Einleitung

Die Vermittlung von Informatik- und Technikenntnissen an Kinder und Jugendliche gewinnt zunehmend an Bedeutung. Plattformen wie die Education-Systeme von LEGO bieten hierfür einen niedrighschwelligen Einstieg in Programmierung und Robotik. Durch vereinfachte Hardware und abstrahierte Softwareumgebungen sind erste praktische Erfahrungen auch ohne umfangreiche Vorkenntnisse möglich.

Mit zunehmender Nutzung entsteht häufig der Wunsch, solche Systeme über ihren ursprünglichen Funktionsumfang hinaus zu erweitern. Insbesondere zusätzliche Hardwarekomponenten eröffnen neue Möglichkeiten, etwa bei der Visualisierung und der Interaktion. Displays mit Touch-Funktionalität sind in diesem Zusammenhang eine naheliegende Erweiterung, da sie eine direkte und intuitive Bedienung ermöglichen.

Die Erweiterung bestehender Embedded-Systeme um externe Komponenten ist allerdings nicht trivial. Neben der reinen Hardwareanbindung spielen vor allem die Kommunikation und die Integration in die bestehende Software eine wichtige Rolle. Die beteiligten Komponenten müssen koordiniert zusammenarbeiten, wobei die vorhandenen Schnittstellen und Laufzeitumgebungen oft Einschränkungen mit sich bringen.

Vor diesem Hintergrund beschäftigt sich die vorliegende Arbeit mit der Frage, wie sich externe Peripherie an ein solches System anbinden lässt, wenn die verfügbare Verbindung dafür nur eingeschränkt geeignet ist. Im Mittelpunkt steht dabei weniger die Ansteuerung eines einzelnen Geräts als eine Softwareschicht, die zwischen Anwendung und Verbindung vermittelt und die Eigenheiten der Übertragung vor der Anwendung verbirgt. Als Anwendungsfall dient die Anbindung eines **Thin-Film-Transistor (TFT)**-Touchscreen-Displays, an dem der Entwurf entwickelt, umgesetzt und bewertet wird.

1.1 Problemstellung

Die Anbindung zusätzlicher Peripherie an eine solche Plattform wird durch drei technische Einschränkungen erschwert. Sie treten gemeinsam auf und verstärken sich gegenseitig.

Erstens ist die Verbindung zwischen den Systemkomponenten anfragegetrieben. Sie ist zwar in beide Richtungen nutzbar, eine Übertragung kann aber nur die übergeordnete Seite anstoßen. Die untergeordnete Seite antwortet ausschließlich. Zweitens ist die je Paket übertragbare Datenmenge sehr klein und fest vorgegeben, sodass Nachrichten üblicher Größe nicht in ein einzelnes Paket passen. Drittens stehen stark beschränkte Laufzeitumgebungen zur Verfügung, die zumindest auf einer Seite weder einen vollständigen Netzwerkstack noch die üblichen Mittel nebenläufiger Programmierung bereitstellen.

Diese Rahmenbedingungen stehen im Gegensatz zu den Anforderungen interaktiver Benutzeroberflächen. Diese setzen eine flexible Übertragung strukturierter Daten und eine zeitnahe Rückmeldung von Benutzereingaben voraus. Gerade die Verarbeitung von Eingaben und die Übertragung zusammengesetzter Zustände sind unter diesen Bedingungen eine Herausforderung.

Daraus ergibt sich die zentrale Problemstellung dieser Arbeit: Wie lässt sich eine Middleware entwerfen, die entfernte Objekte über eine anfragegetriebene Verbindung mit sehr kleiner Paketgröße auf beschränkten Laufzeitumgebungen transparent nutzbar macht?

Die Frage ist dabei nicht an eine bestimmte Plattform gebunden. Die Kombination aus anfragegetriebener Übertragung, kleiner Paketgröße und beschränkter Laufzeitumgebung tritt in verschiedenen Ausprägungen auf, wie Abschnitt 3.5 zeigt.

1.2 Zielsetzung der Arbeit

Ziel dieser Arbeit ist der Entwurf einer Middleware, die entfernte Objekte über eine Verbindung der beschriebenen Art transparent nutzbar macht.

Der Entwurf soll die drei genannten Einschränkungen auf wenige Schichten eingrenzen, sodass sie oberhalb davon nicht mehr sichtbar sind. Dazu gehören:

- die Zerlegung von Nachrichten in Pakete und deren Rahmung,
- ein anfragegetriebener Rückkanal für Ergebnisse und Ereignisse
- sowie eine Abstraktion, die entfernte Objekte wie lokale erscheinen lässt.

Ein weiterer Schwerpunkt liegt darauf, den Entwurf vom konkreten Übertragungsweg zu lösen. Er soll dadurch auf andere Verbindungen derselben Klasse übertragbar bleiben.

Die Bewertung stützt sich auf drei Kriterien. Das erste ist der funktionale Nachweis, dass sich eine interaktive Oberfläche über die entwickelte Abstraktion umsetzen lässt. Das zweite ist der Übertragungsaufwand je Aufruf, gemessen in benötigten Paketen und in der daraus folgenden Verzögerung. Das dritte ist der Ressourcenverbrauch auf beiden Seiten der Verbindung.

1.3 Aufbau der Arbeit

Die vorliegende Arbeit ist wie folgt aufgebaut.

Kapitel 2 führt die Grundlagen zum entfernten Prozeduraufruf, zur Serialisierung und zur Kommunikation eingebetteter Systeme ein. Anschließend ordnet es die Arbeit in bestehende Ansätze ein und benennt die Lücke, die sie schließt.

Kapitel 3 analysiert das Zielsystem und leitet die Anforderungen an die Display-Integration und an die Kommunikation ab. Den Anforderungen werden die Einschränkungen der Plattform gegenübergestellt. Das Kapitel schließt mit einer Verallgemeinerung des Problemraums.

Kapitel 4 beschreibt die Architektur mit ihrem Schichtenmodell, den wiederkehrenden Rollen im Entwurf und dem Kommunikationsmodell. Kapitel 5 vertieft den Entwurf der Transport- und der Kommunikationsschicht. Beide Kapitel bleiben auf der konzeptionellen Ebene.

Kapitel 6 stellt die Referenzimplementierung vor und bildet den Entwurf auf die konkrete Hardware, die Laufzeitumgebungen und den Bus ab. Kapitel 7 bewertet die Lösung hinsichtlich Funktionalität, Übertragungsaufwand und Ressourcenverbrauch. Kapitel 8 diskutiert die Grenzen des Ansatzes und seine Übertragbarkeit auf andere Verbindungen und fasst die Ergebnisse zusammen.

2 Grundlagen und verwandte Arbeiten

Die in dieser Arbeit entwickelte Middleware baut auf bekannten Ansätzen für entfernte Aufrufe und für die Kommunikation in verteilten und eingebetteten Systemen auf. Die ersten drei Abschnitte führen die Begriffe ein, die in den Entwurfskapiteln benötigt werden. Anschließend wird die eigene Lösung in dieses Umfeld eingeordnet. Da das Feld sehr breit ist, werden ausgewählte Vertreter betrachtet und anhand einheitlicher Kriterien verglichen. Ziel ist es, die Lücke zu benennen, die die vorliegende Arbeit schließt.

2.1 Entfernter Prozeduraufruf und Middleware

Bei einem entfernten Prozeduraufruf, englisch [Remote Procedure Call \(RPC\)](#), ruft ein Programm eine Prozedur auf, die auf einem anderen Rechner ausgeführt wird. Im Quelltext sieht dieser Aufruf wie ein gewöhnlicher lokaler Aufruf aus. Die Idee wurde von Birrell und Nelson beschrieben und bildet seitdem die Grundlage der meisten Middleware-Systeme [1]. Der Aufruf wird auf der aufrufenden Seite von einem Stellvertreter entgegengenommen, der dieselbe Signatur besitzt wie die eigentliche Prozedur, sie aber nicht ausführt. Er verpackt den Namen der Prozedur und die Argumente zu einer Nachricht und übergibt sie dem Transport. Auf der ausführenden Seite nimmt ein Gegenstück die Nachricht entgegen, liest die Argumente aus, ruft die eigentliche Prozedur auf und schickt deren Rückgabewert zurück. Dieses Gegenstück wird klassisch als Skelett bezeichnet. Das Umwandeln von Argumenten und Rückgabewerten in eine übertragbare Form heißt Marshalling.

Das Ziel dieses Aufbaus ist die Aufrufstransparenz. Der Aufrufer soll nicht wissen müssen, ob eine Prozedur lokal oder entfernt ausgeführt wird. Vollständig erreichen lässt sich das allerdings nicht. Ein entfernter Aufruf dauert um Größenordnungen länger als ein lokaler, er kann teilweise fehlschlagen, während der Aufrufer weiterläuft, und Zeiger in den lokalen Speicher haben auf der Gegenseite keine Bedeutung. Waldo et al. zeigen, dass sich diese Unterschiede nicht dauerhaft verbergen lassen und in der Schnittstelle sichtbar bleiben sollten [2]. Transparenz wird deshalb meist als Aussage über die Aufrufsyntax verstanden und weniger als Zusicherung gleicher Laufzeiteigenschaften.

Die Schicht, die Stellvertreter, Marshalling und Transport bereitstellt, wird als Middleware bezeichnet. Sie liegt zwischen der Anwendung und der Verbindung, bietet der Anwendung eine einheitliche Schnittstelle an und verbirgt die Eigenheiten der Übertragung [3]. Ein häufig verwendeter Aufbau für solche Systeme ist das Broker-Muster. Es beschreibt den Stellvertreter auf der Client-Seite, den vermittelnden Broker und das Gegenstück auf der Server-Seite als eigenständige Rollen [4].

Der ursprüngliche Ansatz kennt nur einen festen Satz von Prozeduren. Objektorientierte Middleware erweitert ihn auf entfernte Objekte, die zur Laufzeit entstehen und über eine Referenz angesprochen werden. Der Stellvertreter entspricht dann dem Proxy-Muster [5]. Er implementiert dieselbe Schnittstelle wie das entfernte Objekt, führt dessen Referenz mit und gibt jeden Aufruf zusammen mit dieser Referenz weiter. Bekannte Vertreter dieses Ansatzes sind [Common Object Request Broker Architecture \(CORBA\)](#) [6] und Java [Remote Method Invocation \(RMI\)](#) [7]. Die Aufteilung in einen Stellvertreter auf der aufrufenden und ein Gegenstück auf der ausführenden Seite findet sich in Abschnitt 4 wieder, wo das Gegenstück als Adapter bezeichnet wird.

2.2 Serialisierung von Nachrichten

Eine Nachricht liegt im Programm als strukturierter Wert vor, etwa als Objekt, als Liste oder als Abbildung. Übertragen wird dagegen eine Folge von Bytes. Die interne Darstellung eines Werts im Arbeitsspeicher lässt sich dafür nicht direkt verwenden, da sie Verweise enthält und von Wortbreite, Byte-Reihenfolge und Laufzeitumgebung abhängt. Serialisierung bezeichnet die Umwandlung eines strukturierten Werts in eine Bytefolge, Deserialisierung den umgekehrten Weg.

Die gebräuchlichen Formate unterscheiden sich vor allem in zwei Punkten. Der erste ist die Kodierung. Textbasierte Formate wie die [JavaScript Object Notation \(JSON\)](#) oder [Extensible Markup Language \(XML\)](#) sind ohne Werkzeuge lesbar und erleichtern damit die Fehlersuche, benötigen aber mehr Platz, weil Schlüssel und Strukturzeichen als Klartext mit übertragen werden. Binäre Formate stellen dieselben Werte deutlich kürzer dar, ihre Ausgabe ist ohne Kenntnis des Formats jedoch nicht mehr zu deuten. Der zweite Punkt ist die Bindung an ein Schema. Schemafreie Formate führen die Struktur in der Nachricht selbst mit, sodass Sender und Empfänger nichts vorab vereinbaren müssen. Schemagebundene Formate setzen eine geteilte Beschreibung der Datenstruktur voraus und sparen dafür die Feldnamen in der Nachricht ein. Welche Vertreter für ein Gerät mit sehr kleinen Paketen in Frage kommen, behandelt Abschnitt 2.6.

Für diese Arbeit ist eine weitere Eigenschaft binärer Formate wichtig. Ihre Ausgabe kann jedes beliebige Byte enthalten, auch das Nullbyte. Ein Empfänger kann die Länge der Nutzlast deshalb nicht aus dem Inhalt ableiten, etwa indem er Nullbytes am Ende eines Pakets als Füllung abschneidet. Die Rahmung muss die Länge dann ausdrücklich mitführen. Bei einem textbasierten Format fällt dieser Unterschied nicht auf, weil dessen Ausgabe kein Nullbyte enthält. Er zeigt sich erst beim Wechsel auf eine binäre Kodierung. Abschnitt 5.2.3 greift das auf.

2.3 Eingebettete Systeme und ihre Kommunikation

Eingebettete Systeme verfügen über erheblich weniger Speicher und Rechenleistung als ein Arbeitsplatzrechner. [Request for Comments \(RFC\) 7228](#) fasst solche Geräte unter dem Begriff *constrained nodes* zusammen und ordnet sie nach verfügbarem Arbeits- und Programmspeicher in Klassen ein [8]. Die betrachteten Größenordnungen liegen bei einigen zehn bis einigen hundert Kilobyte Arbeitsspeicher. In diesem Rahmen werden Entwurfsentscheidungen wichtig, die auf einem gewöhnlichen Rechner kaum eine Rolle spielen.

Auf solchen Geräten kommen angepasste Laufzeitumgebungen zum Einsatz. MicroPython ist eine für Mikrocontroller ausgelegte Implementierung von Python [9]. Der Sprachkern bleibt weitgehend erhalten, die Standardbibliothek ist dagegen stark verkleinert. Welche Teile vorhanden sind, hängt vom jeweiligen Port ab. Auf stark beschränkter Hardware fehlen häufig ein vollständiger Netzwerkstack und die üblichen Mittel nebenläufiger Programmierung. Ohne Netzwerkstack stehen auch die darauf aufbauenden Abstraktionen nicht zur Verfügung, von Sockets bis zu fertigen Protokollbibliotheken. Ohne Nebenläufigkeit gibt es keinen Hintergrundprozess, der Daten entgegennimmt, während die Anwendung weiterläuft. Hinzu kommt der kleine Heap. Jede Zuweisung erhöht den Druck auf die Speicherbereinigung, weshalb Puffer nach Möglichkeit vorab angelegt und wiederverwendet werden.

Die Verbindung zwischen einem Mikrocontroller und seiner Peripherie erfolgt über Bussysteme, die für kurze Strecken und wenige Leitungen ausgelegt sind. Viele von ihnen arbeiten nach dem Master-Slave-Prinzip. Eine Seite, der Master, stößt jede Übertragung an, die andere Seite antwortet ausschließlich. [Inter-Integrated Circuit \(I2C\)](#) [10] und [Serial Peripheral Interface \(SPI\)](#) [11] sind verbreitete Vertreter, auf Feldebene folgt Modbus im [Remote Terminal Unit \(RTU\)](#)-Modus demselben Muster [12]. Ein Slave, bei dem Daten anfallen, kann diese nicht von sich aus schicken. Er hält sie bereit, bis der Master sie abfragt. Dieses wiederholte Abfragen wird als Polling bezeichnet. Wie schnell ein Ereignis auf der Slave-Seite beim Master ankommt, hängt damit vom Abfragetakt des Masters ab und nicht vom Zeitpunkt, an dem es entstanden ist.

Die je Übertragung nutzbare Datenmenge ist auf solchen Bussen klein und häufig fest vorgegeben. Die größte in einem Paket übertragbare Nutzlast wird als [Maximum Transmission Unit \(MTU\)](#) bezeichnet. Ist eine Nachricht länger, muss sie in mehrere Pakete zerlegt werden. Damit der Empfänger sie wieder zusammensetzen kann, trägt jedes Paket zusätzliche Steuerinformationen, die üblicherweise als Header vorangestellt werden. Diese Steuerinformationen gehen von der ohnehin knappen Nutzlast ab, sodass ihre Größe bei kleinen Paketen unmittelbar auf die Anzahl der nötigen Übertragungen durchschlägt. Wie sich diese Eigenschaften am betrachteten Zielsystem konkret darstellen, zeigt Abschnitt 3.4.

2.4 Einordnung und Auswahlkriterien

Für den Vergleich werden Kriterien herangezogen, die sich direkt aus den Rahmenbedingungen des Zielsystems ergeben. Das Zielsystem nutzt einen Master-Slave-Bus mit sehr kleinen, festen Paketen und läuft in stark eingeschränkten Laufzeitumgebungen.

Folgende Achsen werden betrachtet:

- **Abstraktionsgrad.** Bietet der Ansatz nur den Transport von Nachrichten, einen entfernten Prozeduraufruf oder darüber hinaus die Abstraktion entfernter Objekte?
- **Serialisierung.** Werden Daten textbasiert oder binär kodiert, und ist dafür ein vorab geteiltes Schema nötig?
- **Transportbindung.** Ist der Ansatz an einen konkreten Übertragungsweg gebunden oder von diesem unabhängig?
- **Fragmentierung.** Bringt der Ansatz Mechanismen für die Rahmung und das Zerlegen großer Nachrichten in kleine Pakete mit?
- **Eignung für einen Master-Slave-Bus.** Setzt der Ansatz voraus, dass die untergeordnete Seite eigenständig senden kann, oder kommt er mit einem rein anfragegetriebenen Kanal aus?
- **Ressourcenbedarf.** Lässt sich der Ansatz innerhalb der Grenzen von MicroPython und Pybricks betreiben?

Die betrachteten Ansätze liegen dabei nicht alle auf derselben Ebene. Einige sind vollständige Rahmenwerke, andere beschränken sich auf die Beschreibung eines Nachrichtenformats. Wieder andere behandeln nur die Kodierung von Daten oder die Anbindung an einen bestimmten Bus. Ein Vergleich, der diese Unterschiede außer Acht lässt, stellt Ansätze nebeneinander, die sich gegenseitig gar nicht ersetzen können. Im abschließenden Vergleich wird die Ebene deshalb in einer eigenen Spalte ausgewiesen.

Diese Kriterien strukturieren die folgende Betrachtung und münden in den abschließenden Vergleich in Tabelle I.

2.5 Etablierte Rahmenwerke für entfernte Aufrufe

Entfernte Prozeduraufrufe wurden früh standardisiert. Der [Open Network Computing Remote Procedure Call \(ONC RPC\)](#) [13], [14] legt ein Protokoll und mit der External Data Representation ein eigenes Serialisierungsformat fest [15]. Der Ansatz ist auf leistungsfähigere Netzwerke ausgelegt und nicht für Kanäle mit Paketgrößen im Bereich weniger Byte gedacht.

JSON-RPC verfolgt eine deutlich schlankere Idee [16]. Die Spezifikation beschreibt ausschließlich das Format der Anfrage- und Antwortnachrichten auf Basis der [JSON](#) und überlässt den Transport bewusst der Anwendung. Diese Trennung ist dem hier verfolgten Ansatz ähnlich. Allerdings setzt JSON-RPC voraus, dass eine darunterliegende Schicht vollständige Nachrichten zustellen kann. Es definiert weder eine Rahmung noch eine Fragmentierung für sehr kleine Pakete und sieht keinen Mechanismus für einen anfragegetriebenen Rückkanal auf einem Master-Slave-Bus vor.

Am anderen Ende des Spektrums steht [gRPC Remote Procedure Call \(gRPC\)](#) [17]. Das Rahmenwerk verbindet Protocol Buffers als schemagebundene Serialisierung mit [Hypertext Transfer Protocol \(HTTP\)/2](#) als Transport und unterstützt unter anderem Datenströme in beide Richtungen. Diese Leistungsfähigkeit erfordert jedoch einen vollständigen [HTTP/2](#)-Stack und setzt eine bidirektionale Verbindung voraus. Für den betrachteten LEGO-Bus und die eingesetzten Laufzeitumgebungen ist [gRPC](#) zu schwergewichtig und zu stark an seinen Transport gebunden.

Zusammengefasst setzen die etablierten Rahmenwerke entweder einen leistungsfähigen Transport voraus oder standardisieren nur die Nachrichtenebene. Rahmung, Fragmentierung und der Rückkanal bleiben dann offen. Genau diese darunterliegende Schicht behandelt die vorliegende Arbeit.

2.6 Serialisierung für ressourcenarme Geräte

Da die Serialisierung die Nachrichtengröße und damit die Anzahl der nötigen Pakete bestimmt, ist sie für ein System mit kleinen Paketen besonders relevant. Die **JSON** ist schemafrei und gut lesbar, erzeugt aber durch Klartextschlüssel und Strukturzeichen vergleichsweise große Ausgaben [18]. Binäre und schemafreie Formate wie MessagePack [19] und die **Concise Binary Object Representation (CBOR)** [20], [21] erreichen bei gleichem Datenmodell deutlich kompaktere Darstellungen. Schemagebundene Formate wie Protocol Buffers gehen noch einen Schritt weiter und setzen eine geteilte Beschreibung der Datenstruktur voraus [22]. Für sehr kleine Geräte existiert mit Nanopb [23] eine ressourcenschonende Implementierung dieses Ansatzes.

In der vorliegenden Arbeit wurde zunächst die **JSON** verwendet. Dafür gab es zwei praktische Gründe. Zum einen steht das Format in beiden Laufzeitumgebungen ohne zusätzliche Abhängigkeit zur Verfügung. Zum anderen ist die Nutzlast im Klartext lesbar. Auf einem Bus, dessen Datenverkehr sich nur mit erheblichem Aufwand mitschneiden lässt, ist das bei der Fehlersuche ein deutlicher Vorteil. Solange die übrigen Schichten noch entwickelt wurden, war diese Lesbarkeit wichtiger als eine kompakte Kodierung.

Mit zunehmender Reife der Umsetzung änderte sich diese Gewichtung. Bei einer Nutzlast von wenigen Byte je Paket wirkt sich jedes zusätzliche Zeichen direkt auf die Anzahl der zu übertragenden Pakete und damit auf die Dauer eines Aufrufs aus. Im weiteren Verlauf wurde deshalb auf MessagePack [19] gewechselt. Das Format bildet dasselbe Datenmodell ab, kommt ebenfalls ohne vorab geteiltes Schema aus und stellt die Daten binär und damit deutlich kürzer dar. Die Lesbarkeit geht dabei verloren. Sie lässt sich bei der Fehlersuche aber zurückgewinnen, indem auf die textbasierte Kodierung umgeschaltet wird, da beide Formate hinter derselben Schnittstelle liegen.

Das Kodieren und Dekodieren der Nutzlast ist dafür an einer einzigen Stelle gebündelt. Ein Wechsel der Kodierung betrifft diese Stelle und die Rahmung, die dafür eine ausdrückliche Längenangabe je Paket benötigt. Der entfernte Aufruf und das Objektmodell bleiben unverändert. Der beschriebene Wechsel zeigt zugleich, dass diese Trennung in der Praxis funktioniert. Die Evaluation vergleicht beide Kodierungen und gibt den Overhead in benötigten Paketen je Aufruf an.

2.7 Protokolle für eingebettete Systeme und Sensornetze

Neben den allgemeinen Rahmenwerken existieren Protokolle, die näher an der betrachteten Domäne liegen. Firmata [24] ermöglicht die Steuerung eines Mikrocontrollers von einem übergeordneten Rechner aus und überträgt dazu einen festgelegten Satz von Befehlen über eine serielle Verbindung. Die Aufteilung in einen steuernden Rechner und ein reagierendes Gerät ähnelt der hier vorliegenden Situation. Firmata beschränkt sich allerdings auf einen vordefinierten Befehlsumfang und bietet weder einen allgemeinen entfernten Aufruf noch eine Abstraktion entfernter Objekte.

Message Queuing Telemetry Transport (MQTT) [25] ist auf ressourcenarme Geräte und kleine Pakete ausgelegt und kommt ohne eine durchgehende **Transmission Control Protocol (TCP)**-Verbindung aus. Es folgt jedoch einem Modell aus Veröffentlichen und Abonnieren, benötigt ein Gateway sowie einen Vermittler und setzt voraus, dass das Gerät eigenständig veröffentlichen kann. Auf einem reinen Master-Slave-Bus, auf dem die untergeordnete Seite nicht von sich aus senden darf, ist dieses Modell nicht unmittelbar anwendbar.

Diese Protokolle liegen nah an der betrachteten Domäne, lösen aber eine andere Aufgabe. Sie zielen auf Gerätesteuerung mit festem Befehlssatz oder auf nachrichtenbasiertes Veröffentlichen und Abonnieren. Einen allgemeinen, objektorientierten entfernten Aufruf über einen anfragegetriebenen Kanal bieten sie nicht.

2.8 Der betrachtete Bus und PUPRemote als Transportbasis

Der Hub und die angeschlossenen Geräte verkehren über **LEGO Power Functions 2.0 (LPF2)**, ein Protokoll, das auf einer **Universal Asynchronous Receiver Transmitter (UART)**-Verbindung über die Sensorports aufsetzt [26]. Der Hub ist dabei der Master und stößt jede Übertragung an. Ein Gerät meldet sich beim Anstecken mit den Modi an, die es unterstützt. Jeder Modus beschreibt einen Kanal mit festem Namen, fester Länge und festem Datentyp. Der Hub wählt einen dieser Modi aus und liest dessen Wert oder schreibt einen Wert hinein.

PUPRemote [27] nimmt in diesem Umfeld eine Sonderstellung ein, da es in dieser Arbeit als Baustein der eigenen Lösung eingesetzt wird. Die Bibliothek erschließt den **LEGO Powered Up (PUP)**- und **LPF2**-Bus und stellt darauf benannte Aufrufe bereit. Jeder Aufruf belegt einen eigenen Modus des Busses und ist über feste Formatangaben für beide Richtungen beschrieben. Der Hub schreibt die Argumente in den zugehörigen Modus und liest anschließend den Rückgabewert aus demselben Modus zurück. Mit PUPRemote werden auf beiden Seiten Befehle registriert, die anschließend vom Master aufgerufen werden können. Die Länge der Befehlsnamen, der Argumente und der Rückgabewerte ist dabei durch die festen Formatangaben begrenzt. PUPRemote besitzt damit durchaus eine eigene Aufrufsemantik und ist nicht auf reinen Datentransport beschränkt.

Diese Semantik trägt jedoch nur so weit, wie sich ein Aufruf auf einen vorab registrierten Modus und ein einzelnes Paket fester Größe abbilden lässt. Die Anzahl der Modi ist durch den Bus begrenzt, die Formate stehen zur Übersetzungszeit fest, und für Nachrichten, die größer sind als ein Paket, ist kein Mechanismus vorgesehen. Eine Middleware, die beliebige Methoden auf einer zur Laufzeit wachsenden Menge entfernter Objekte anbieten soll, kann daher nicht einen Modus je Aufruf vergeben.

Die vorliegende Arbeit nutzt PUPRemote deshalb nur als Busanbindung. Registriert wird ein einziges Kommando, dessen Nutzlast aus Sicht von PUPRemote ein Block fester Länge ohne Struktur ist. Die Zerlegung von Nachrichten in Pakete, deren Rahmung, die Zuordnung von Antworten zu Anfragen, der anfragegetriebene Rückkanal sowie das Objektmodell aus Stellvertretern und serverseitigen Adaptern liegen vollständig darüber. PUPRemote übernimmt damit die Rolle der Busanbindung und kann durch andere Bibliotheken ersetzt werden, die denselben Austausch von Blöcken leisten.

2.9 Abgrenzung

Tabelle I fasst die betrachteten Ansätze entlang der eingangs definierten Kriterien zusammen.

Tabelle I: Vergleich der betrachteten Ansätze entlang der Auswahlkriterien

Ansatz	Ebene	Abstraktionsgrad	Serialisierung	Transportbindung	Fragmentierung	Eignung kleiner Master-Slave-Bus
gRPC	Middleware	RPC und Datenströme	Protocol Buffers, schemagebunden	an HTTP/2 gebunden	über HTTP/2	ungeeignet, setzt Push voraus
JSON-RPC	Nachrichtenformat	nur Nachrichtenformat	JSON, textbasiert	unabhängig, offen	nicht definiert	offen, Rückkanal fehlt
Message-Pack und CBOR	Serialisierung	nur Serialisierung	binär, schemafrei	entfällt	entfällt	entfällt
Firmata	Geräteprotokoll	Gerätesteuerung mit festen Befehlen	binär, fest	seriell, fest	nein	passend, aber fester Befehlssatz
MQTT	Nachrichtenprotokoll	Veröffentlichen und Abonnieren	binär, kompakt	unabhängig, braucht Gateway	teilweise	setzt eigenständiges Senden voraus
PUPRemote	Transportanbindung, hier als Baustein verwendet	entfernter Aufruf, ein Modus je Aufruf	feste Formatangaben je Modus	an PUP und LPF2 gebunden	nein, ein Paket	passend, aber nur ein Paket
Diese Arbeit	Middleware	entfernter Aufruf und entfernte Objekte	MessagePack, gekapselt und austauschbar	gekapselt, PUPRemote als erste Bindung	ja, eingebaut	passend, anfragegetriebener Rückkanal

Aus dem Vergleich wird die Lücke deutlich. Die leistungsfähigen Rahmenwerke setzen einen Transport voraus, den das Zielsystem nicht bietet. Die schlanken Ansätze standardisieren nur Teilaspekte und lassen Rahmung, Fragmentierung und den Rückkanal offen. Die domänennahen Protokolle lösen eine andere Aufgabe oder setzen voraus, dass die untergeordnete Seite eigenständig senden kann. PUPRemote liegt schließlich eine Ebene tiefer. Es erschließt den Bus und wird in dieser Arbeit als Busanbindung genutzt, ersetzt aber nicht die gesuchte Middleware.

Die vorliegende Arbeit verbindet diese Aspekte zu einer zusammenhängenden Lösung. Sie stellt einen transportunabhängig entworfenen entfernten Aufruf samt Objektmodell bereit, bringt Rahmung und Fragmentierung für sehr kleine Pakete mit und setzt einen anfragegetriebenen Rückkanal für einen Master-Slave-Bus um. All das läuft innerhalb der Grenzen von MicroPython und Pybricks. Der Beitrag liegt damit weniger in einem einzelnen neuen Baustein als in der Kombination bekannter Konzepte für einen bisher nicht abgedeckten Anwendungsfall.

3 Systemanalyse und Anforderungen

3.1 Zielsystem und Hardwareumgebung

Ausgangspunkt der vorliegenden Arbeit ist eine bestehende Hardwareplattform aus dem LEGO-Education-Umfeld. Sie legt die technischen Rahmenbedingungen fest, in denen die in Abschnitt 1 motivierte Display-Integration umgesetzt werden muss. Im Folgenden wird zunächst die zentrale Hardwarekomponente vorgestellt. Anschließend werden ihre Eigenschaften im Hinblick auf die Anbindung eines Touchscreen-Displays analysiert. Aus dieser Analyse ergibt sich die Notwendigkeit einer externen Erweiterung.

3.1.1 Der LEGO Education SPIKE Prime Hub als Ausgangsplattform

Den Kern des Zielsystems bildet der LEGO Education SPIKE Prime Hub [28]. Hierbei handelt es sich um einen programmierbaren Steuerbaustein, der primär für den Einsatz im Bildungsbereich konzipiert ist und über sechs Ein- und Ausgabeports zum Anschluss von LEGO-kompatiblen Sensoren und Aktoren verfügt. Für die Interaktion mit dem Nutzer stellt der Hub eine Matrix aus 5×5 **Light Emitting Diodes (LEDs)**, drei Tasten und einen einfachen Lautsprecher bereit.

Intern basiert der Hub auf einem STM32F413-Mikrocontroller [29] mit einem ARM-Cortex-M4-Kern, einer Taktfrequenz von etwa 100 MHz, rund 1 bis 1,5 MB Flash-Speicher sowie 320 kB **Static Random Access Memory (SRAM)** [30]. Die Programmierung kann grafisch über die offizielle LEGO-Software oder textbasiert mittels MicroPython erfolgen. Welche Firmware konkret eingesetzt wird und wie sich diese Wahl auf die spätere Lösung auswirkt, wird in Abschnitt 3.4 vertieft.

Für die vorliegende Arbeit sind drei Eigenschaften des Hubs von besonderer Bedeutung:

- Der Hub stellt ausschließlich passive, an die Sensor- und Motorports gebundene Erweiterungsmöglichkeiten bereit. Eine direkte Anbindung beliebiger Peripheriegeräte über offene Schnittstellen wie **SPI** [11] oder **I2C** [10] ist nicht vorgesehen.
- Die verfügbaren Anzeige- und Eingabemöglichkeiten beschränken sich auf die genannte **LED-Matrix** sowie die Hub-Tasten.
- Rechenleistung und Arbeitsspeicher sind für einen Mikrocontroller zwar typisch, für grafische Benutzeroberflächen jedoch stark begrenzt.

Diese Eigenschaften bilden den Rahmen, in dem die Display-Integration entworfen werden muss.

3.1.2 Notwendigkeit einer externen Erweiterung

Aus den in Abschnitt 1.1 motivierten Anforderungen an eine grafische Benutzeroberfläche mit Touch-Funktionalität ergibt sich ein direkter Konflikt mit der beschriebenen Plattform. Die im Hub integrierten Ausgabe- und Eingabemöglichkeiten beschränken sich auf eine niedrig auflösende **LED-Matrix** und einfache Tasten und sind damit für die Darstellung komplexer Inhalte sowie für eine differenzierte Benutzerinteraktion ungeeignet. Eine Erweiterung des Hubs selbst ist ebenfalls nicht vorgesehen, da er als geschlossene Komponente konzipiert ist und keine Schnittstellen zum direkten Anschluss grafischer Peripherie bereitstellt.

Die geforderte Display-Funktionalität lässt sich daher nur außerhalb des Hubs umsetzen. Die Ansteuerung des Displays muss an eine externe Hardwarekomponente abgegeben werden, die an einen der vorhandenen Sensorports des Hubs angeschlossen wird. Hieraus ergeben sich zwei eigenständige Teilprobleme, die in den folgenden Abschnitten weiter ausgearbeitet werden:

- die Wahl einer geeigneten externen Hardware zur Ansteuerung eines Displays sowie
- eine geeignete Kommunikation zwischen dem Hub und dieser externen Komponente.

3.2 Anforderungen an die Display-Integration

3.2.1 Anforderungen an ein externes Display

Bevor konkrete Hardwarekomponenten betrachtet werden, lassen sich aus der Zielsetzung der Arbeit allgemeine Anforderungen an ein externes Display ableiten:

- **D1 – Grafische Darstellung.** Das Display soll die Darstellung einfacher grafischer Elemente wie Schaltflächen, Beschriftungen und Anzeigen ermöglichen.
- **D2 – Berührungseingabe.** Es soll zugleich Benutzereingaben über einen Touchscreen entgegennehmen.
- **D3 – Embedded-typische Auslegung.** Auflösung, Größe und Energieaufnahme orientieren sich an typischen Embedded-Anwendungen. Damit kommen kleine **TFT**-Displays mit einer Auflösung von etwa 240×320 Pixeln in Frage.

Solche Displays sind in unterschiedlichen Ausführungen verfügbar. Im Embedded-Bereich weit verbreitet sind Module mit ILI9341-Display-Controller. Sie unterstützen eine Auflösung von 240×320 Pixeln bei einer Farbtiefe von 18 Bit und werden über **SPI** angesteuert. Für die Erkennung von Berührungen kommen typischerweise kapazitive Touchcontroller wie der FT6X36 zum Einsatz, die über **I2C** angebunden werden und neben Einzelberührungen auch Mehrfingergesten erfassen können. Solche Module sind kostengünstig verfügbar und werden in der Maker- und Embedded-Community häufig in Verbindung mit Mikrocontrollern der ESP32-Familie eingesetzt.

3.2.2 Anforderungen an einen externen Mikrocontroller

Da der SPIKE Prime Hub selbst kein **TFT**-Display über **SPI** anbinden kann, muss zwischen Hub und Display ein zusätzlicher Mikrocontroller vermitteln. Er übernimmt die Ansteuerung des Displays und die Verarbeitung der Touch-Eingaben und kommuniziert gleichzeitig mit dem Hub. Aus dieser Doppelrolle ergeben sich folgende Anforderungen an die Hardware:

- **S1 – Displayschnittstellen.** Sie muss über geeignete Schnittstellen zur Ansteuerung typischer **TFT**-Displays verfügen, insbesondere **SPI** für die Bildschirmdaten und **I2C** für den Touch-Controller.
- **S2 – Rechenleistung und Speicher.** Sie muss ausreichend Rechenleistung und Speicher bieten, um eine grafische Bibliothek wie etwa **Light and Versatile Graphics Library (LVGL)** [31] ausführen zu können.
- **S3 – Busanbindung.** Sie muss über eine Schnittstelle verfügen, die kompatibel zu den Sensorports des SPIKE Prime Hubs ist, oder eine entsprechende Anbindung über **LPF2** unterstützen.

Diese Anforderungen erfüllen Erweiterungsboards aus dem Bildungs- und Maker-Bereich, die speziell für die Anbindung an LEGO-Hubs entworfen wurden. Ein Beispiel sind die ESP32-basierten Boards der LMS-ESP32-Reihe [32]. Sie geben sich gegenüber dem Hub als LEGO-Sensor aus und können dadurch über das **LPF2**-Protokoll kommunizieren. Welche Hardware in dieser Arbeit konkret eingesetzt wird und wie die Software dafür eingerichtet ist, beschreibt Abschnitt 6.

3.2.3 Resultierende Systemstruktur

Aus diesen Überlegungen ergibt sich eine Systemstruktur aus drei Teilen. Der SPIKE Prime Hub führt die Anwendung aus, ein externer Mikrocontroller steuert das Display an, und ein **TFT**-Touchscreen-Display dient zur Anzeige und Eingabe. Hub und Mikrocontroller sind über einen Sensorport des Hubs und das **LPF2**-Protokoll verbunden. Im Folgenden wird der Hub als Steuereinheit bezeichnet. Mikrocontroller und Display bilden zusammen die Anzeigeeinheit. Wie die Kommunikation zwischen beiden aussehen muss und welchen Einschränkungen sie unterliegt, betrachten die folgenden Abschnitte.

3.3 Kommunikationsanforderungen

Der vorige Abschnitt legt fest, aus welchen Komponenten das System besteht. Offen ist noch, was die Verbindung zwischen Steuer- und Anzeigeeinheit leisten muss. Die folgenden Anforderungen leiten sich aus dem Anwendungsfall einer interaktiven Oberfläche ab und beziehen sich nicht auf einen bestimmten Übertragungsweg. Sie sind mit K1 bis K6 nummeriert, damit Abschnitt 7 später auf sie verweisen kann.

- **K1 – Strukturierte Nachrichten unbestimmter Länge.** Ein Bedienelement lässt sich nicht durch einen einzelnen Wert beschreiben. Zum Erzeugen einer Schaltfläche gehören Position, Abmessungen und Beschriftung. Ein Beschriftungstext hat keine feste Länge, und eine Auswahlliste enthält unterschiedlich viele Einträge. Die Verbindung muss deshalb zusammengesetzte Werte übertragen können, deren Länge erst zur Laufzeit feststeht. Feste Formatangaben je Aufruf, wie sie die in Abschnitt 2 betrachtete Transportbibliothek verwendet, reichen dafür nicht aus.
- **K2 – Aufrufe mit zuordenbarem Ergebnis.** Viele Aufrufe liefern einen Rückgabewert. Das Erzeugen eines Objekts liefert eine Referenz, über die es später angesprochen wird. Das Auslesen eines Zustands liefert den entsprechenden Wert. Zu jeder Anfrage muss also eine Antwort zurückkommen. Werden mehrere Anfragen nacheinander abgesetzt, muss erkennbar bleiben, welche Antwort zu welcher Anfrage gehört.
- **K3 – Rückfluss von Ereignissen.** Berührungen entstehen auf der Anzeigeeinheit, ausgewertet werden sie aber von der Anwendung auf der Steuereinheit. Informationen müssen daher auch dann von der Anzeige- zur Steuereinheit gelangen, wenn sie keine Antwort auf einen Aufruf sind. Ereignisse sollen dabei nicht verloren gehen. Lässt sich ein Verlust nicht vermeiden, muss er zumindest erkennbar sein.
- **K4 – Antwortzeit.** Berührung und Reaktion auf dem Bildschirm sollen für den Benutzer zusammengehören. Als Richtwert dient die bei direkter Manipulation übliche Grenze von etwa 100 Millisekunden. Gemeint ist dabei der gesamte Vorgang aus Erkennung, Übertragung, Verarbeitung und Rückmeldung, nicht die Übertragung eines einzelnen Pakets.
- **K5 – Begrenzter Speicherbedarf.** Auf beiden Seiten der Verbindung arbeitet ein Mikrocontroller. Puffer brauchen deshalb eine feste Obergrenze, die schon beim Entwurf bekannt ist. Verfahren, die im ungünstigen Fall beliebig viel Speicher belegen, etwa das Sammeln aller bisher aufgetretenen Ereignisse, kommen nicht in Frage.
- **K6 – Unabhängigkeit vom Übertragungsweg.** Der Übertragungsweg ist durch die Plattform vorgegeben und gehört nicht zum eigentlichen Problem. Seine Eigenschaften sollen sich deshalb nur in einem klar abgegrenzten Teil des Entwurfs auswirken. Prüfen lässt sich das daran, wie viel geändert werden müsste, um denselben Dienst über einen anderen Weg anzubieten.

K1 bis K3 betreffen den Inhalt der Übertragung, K4 und K5 die Bedingungen, unter denen sie stattfindet, und K6 den Aufbau des Entwurfs. Der folgende Abschnitt stellt diesen Anforderungen die Eigenschaften der Plattform gegenüber.

3.4 Einschränkungen der Plattform

Steuer- und Anzeigeeinheit sind über einen Sensorport des Hubs und das [LPF2-Protokoll](#) gekoppelt. Aus dieser Kopplung ergeben sich drei Einschränkungen, die den Anforderungen aus dem vorigen Abschnitt entgegenstehen. Sie werden im Folgenden einzeln beschrieben und den betroffenen Anforderungen zugeordnet. Es handelt sich dabei um Eigenschaften der Kopplung selbst und nicht um Schwächen einer bestimmten Umsetzung. Wie der Entwurf darauf reagiert, zeigen Abschnitt 4 und Abschnitt 5. Die konkrete Umsetzung beschreibt Abschnitt 6.

3.4.1 Eingeschränkte Kommunikationsrichtung

Das LPF2-Protokoll arbeitet nach dem Master-Slave-Prinzip. Der Hub ist dabei der Master. Er wählt den aktiven Modus des angeschlossenen Geräts aus und stößt jede Übertragung an. Das Gerät meldet sich als Sensor an und stellt Werte bereit, die der Hub abrufen kann. Von sich aus senden kann es nicht.

Die in Abschnitt 2 beschriebene Bibliothek PUPRemote bildet das direkt ab und bietet ein einziges Zugriffsmuster an. Die Steuereinheit schreibt Daten in einen Modus und liest im selben Vorgang die Antwort zurück. Jede Übertragung besteht damit aus Anfrage und Antwort, und die Anzeigeeinheit antwortet ausschließlich.

Für die Anforderungen bedeutet das, dass sich weder Ergebnisse (K2) noch Ereignisse (K3) zustellen lassen. Beides kann nur bereitgehalten und abgefragt werden. Entsteht ein Ergebnis erst nach der letzten Anfrage, ist eine weitere Anfrage nötig, und eine Berührung zwischen zwei Anfragen muss bis zur nächsten Anfrage zwischengespeichert werden. Wann eine Rückmeldung eintrifft, bestimmt damit die Seite, die sie abholt. Die zeitliche Auflösung von Ereignissen ist dadurch durch den Abfragetakt der Steuereinheit begrenzt, was sich direkt auf K4 auswirkt.

3.4.2 Begrenzte Paketgröße

Ein Modus des Busses überträgt eine feste, vorab vereinbarte Anzahl von Bytes. Zusammen mit der auf dem Hub eingesetzten Firmware liegt die nutzbare Obergrenze bei 16 Byte je Übertragung. Oberhalb dieses Werts treten Prüfsummenfehler auf. Da jedes Paket zusätzlich Steuerinformationen für das Zusammensetzen tragen muss, bleibt für die Nachricht selbst noch weniger übrig. In der Referenzimplementierung sind es 13 nutzbare Byte je Paket, wie Abschnitt 6 zeigt.

Schon eine einfache Aufrufnachricht mit Objektreferenz, Methodenname und wenigen Argumenten überschreitet diesen Rahmen deutlich. K1 lässt sich mit einem einzelnen Paket also nicht erfüllen, und Nachrichten müssen in mehrere Pakete zerlegt werden. Hinzu kommt, dass die Steuerinformationen bei einer Nutzlast von wenigen Byte einen großen Anteil der Übertragung ausmachen.

Eine weitere Beschränkung ergibt sich aus der Stromversorgung. Ein Display benötigt mehr Strom, als die Logikversorgung des Ports liefert. Die Anzeigeeinheit fordert deshalb die höhere Versorgungsspannung über den Port an. In diesem Fall verkürzt sich die zulässige Länge der Modusnamen von elf auf fünf Zeichen, wodurch auch die Namen der Kanäle knapp werden. Abschnitt 6.2 geht auf die Versorgung näher ein.

3.4.3 Beschränkte Laufzeitumgebung

Der Hub wird in einer für Mikrocontroller ausgelegten Python-Variante programmiert, deren Funktionsumfang gegenüber einer vollständigen Laufzeitumgebung deutlich eingeschränkt ist. Konkret handelt es sich um MicroPython [9] in der Ausprägung von Pybricks [33], einer Firmware für LEGO-Hubs. Abschnitt 6.1 geht auf die Unterschiede zwischen beiden Seiten ein.

Ein Netzwerkstack und die darauf aufbauenden Abstraktionen stehen nicht zur Verfügung. Die üblichen Bausteine verteilter Kommunikation lassen sich deshalb nicht verwenden und müssen durch eigene ersetzt werden.

Schwerer wiegt, dass auf der Steuereinheit keine echte Nebenläufigkeit zur Verfügung steht. Es gibt keinen Hintergrundprozess, der Pakete entgegennimmt oder Ergebnisse einsammelt, während die Anwendung weiterläuft. Das Warten auf ein Ergebnis findet deshalb im Kontrollfluss der Anwendung selbst statt. Solange das Ergebnis eines Aufrufs nicht vorliegt, steht die Anwendung still. Mehrere Aufrufe können sich damit nicht überlappen, und ihre Übertragungszeiten addieren sich.

Dazu kommt der begrenzte Arbeitsspeicher auf beiden Seiten. K5 ist auf dieser Plattform damit eine harte Grenze. Puffer für unvollständig empfangene Nachrichten, für bereitgehaltene

Ergebnisse und für zwischengespeicherte Ereignisse brauchen jeweils eine feste Obergrenze. Außerdem muss festgelegt sein, was passiert, wenn diese erreicht wird.

3.4.4 Wechselwirkung der Einschränkungen

Einzelnen betrachtet ließe sich jede der drei Einschränkungen bewältigen. Kleine Pakete lassen sich durch Zerlegung überbrücken, eine fehlende Sendemöglichkeit durch regelmäßiges Abfragen, und ein Ablauf lässt sich auch ohne Nebenläufigkeit sequenziell formulieren. Zusammen verstärken sie sich jedoch gegenseitig.

Durch die kleine Paketgröße zerfällt eine Nachricht in viele Pakete. Wegen der eingeschränkten Kommunikationsrichtung erfordert jedes dieser Pakete eine eigene Anfrage der Steuereinheit samt Antwort, und für das Abholen des Ergebnisses sind weitere Anfragen nötig. Ohne Nebenläufigkeit werden all diese Anfragen nacheinander abgearbeitet.

Für den weiteren Entwurf folgt daraus, dass die Dauer eines Aufrufs vor allem von der Anzahl der nötigen Übertragungen abhängt und weniger von der Geschwindigkeit des Busses. Ein zusätzliches Byte Steuerinformation je Paket kann bei einer Nutzlast von wenigen Byte die Anzahl der Pakete und damit die Dauer eines Aufrufs erhöhen. Ziel des Entwurfs ist deshalb vor allem, Übertragungen einzusparen. Abschnitt 7 greift das auf und beziffert es.

Tabelle II fasst die Gegenüberstellung von Anforderungen und Einschränkungen zusammen.

Tabelle II: Gegenüberstellung der Kommunikationsanforderungen und der Einschränkungen der Plattform

Nr.	Anforderung	Entgegenstehende Einschränkung	Folge für den Entwurf
K1	Strukturierte Nachrichten unbestimmter Länge	Feste Paketgröße von 16 Byte	Nachrichten müssen zerlegt und wieder zusammengesetzt werden
K2	Aufrufe mit zuordenbarem Ergebnis	Keine eigenständige Übertragung durch die Anzeigeeinheit	Ergebnisse werden bereitgehalten und abgefragt, Zuordnung über eine mitgeführte Kennung
K3	Rückfluss von Ereignissen	Keine eigenständige Übertragung durch die Anzeigeeinheit	Ereignisse werden zwischengespeichert und von der Steuereinheit abgeholt
K4	Antwortzeit	Alle drei Einschränkungen gemeinsam	Anzahl der Übertragungen je Aufruf wird zur bestimmenden Größe
K5	Begrenzter Speicherbedarf	Beschränkter Arbeitsspeicher beider Seiten	Feste Obergrenzen für alle Puffer, festgelegtes Verhalten beim Überlauf
K6	Unabhängigkeit vom Übertragungsweg	Eigenheiten von Bus und PUPRemote	Alle wegabhängigen Anteile in einer austauschbaren Schicht bündeln

3.5 Verallgemeinerung des Problemraums

Die im vorigen Abschnitt beschriebenen Einschränkungen wurden am konkreten Zielsystem hergeleitet. Sie sind jedoch nicht an dieses System gebunden. Das ist für die weitere Arbeit wichtig, denn davon hängt ab, ob der folgende Entwurf eine Einzellösung ist oder für eine ganze Klasse von Verbindungen gilt.

Die drei Eigenschaften lassen sich von der Plattform ablösen und als allgemeine Merkmale einer Verbindung formulieren. Erstens ist die Übertragung anfragegetrieben, das heißt, nur eine der beiden Seiten kann eine Übertragung anstoßen. Zweitens ist die Nutzlast je Übertragung klein und fest vorgegeben. Drittens steht auf mindestens einer Seite eine Laufzeitumgebung zur Verfügung, die weder einen vollständigen Netzwerkstack noch die üblichen Mittel nebenläufiger Programmierung bereitstellt.

Diese Kombination tritt über den betrachteten Bus hinaus in verschiedenen Ausprägungen auf. Bei der Kommunikation über **I2C** treibt der Master den Bus, und ein Slave kann eine Übertragung nicht selbstständig beginnen. Modbus **RTU** folgt demselben Muster aus Anfrage und Antwort. Auch bei Bluetooth Low Energy erfolgt der übliche Zugriff über lesende Anfragen

des Clients, und die standardmäßig ausgehandelte Nutzlast liegt in derselben Größenordnung wie beim hier betrachteten Bus. In allen genannten Fällen stellen sich dieselben Fragen. Wie werden Nachrichten zerlegt und wieder zusammengesetzt? Wie gelangen Ergebnisse und Ereignisse zur anfragenden Seite zurück? Und wie lässt sich all das vor der Anwendung verbergen?

Die vorliegende Arbeit behandelt den betrachteten Bus daher als einen Vertreter dieser Klasse. Der Entwurf in den folgenden Kapiteln bezieht sich deshalb auf die genannten Merkmale und nicht auf Eigenschaften einer bestimmten Plattform. Inwieweit das gelungen ist, wird am Ende der Arbeit diskutiert.

4 Architektur des Systems

Dieses Kapitel leitet die Architektur der Lösung aus den Anforderungen des vorigen Kapitels ab. Die Darstellung bleibt auf der konzeptionellen Ebene und nennt weder konkrete Bibliotheken noch Datenformate oder Klassen. Beschrieben werden die Ziele der Architektur, der geschichtete Aufbau mit den Aufgaben der einzelnen Schichten, die wiederkehrenden Rollen innerhalb dieser Schichten sowie das Kommunikationsmodell. Wie sich der Entwurf auf eine konkrete Hardware, Laufzeitumgebung und einen konkreten Bus abbilden lässt, zeigt Abschnitt 6.

4.1 Zielsetzung der Architektur

Die Architektur verfolgt mehrere Ziele, die sich direkt aus den zuvor formulierten Anforderungen ergeben. Im Mittelpunkt steht die Aufruftransparenz auf der Anwendungsseite. Die Anwendung soll mit gewöhnlichen Methodenaufrufen arbeiten und nichts über Pakete, Bestätigungen oder das Abholen von Ergebnissen wissen müssen.

Ein zweites Ziel ist die Unabhängigkeit vom konkreten Übertragungsweg. Die höheren Schichten sollen so entworfen sein, dass der Transport hinter einer einheitlichen Schnittstelle liegt und ausgetauscht werden kann. Hinzu kommen die Bewältigung der kleinen Paketgröße durch Rahmung und Fragmentierung, ein anfragegetriebener Rückkanal für Ergebnisse und Ereignisse sowie ein sparsamer Umgang mit Speicher und Rechenzeit. Schließlich soll die Architektur eine klare und symmetrische Struktur besitzen, in der beide Seiten dieselben Schnittstellen teilen.

Aus diesen Zielen ergibt sich ein geschichteter Aufbau nach dem Vorbild einer Middleware mit entferntem Aufruf. Jede Schicht kapselt eine klar umrissene Aufgabe und stellt der darüberliegenden Schicht eine höhere Abstraktion bereit. Der Vorteil dieses Aufbaus ist, dass sich die schwierigen Eigenschaften des Zielsystems, also die geringe Paketgröße und die eingeschränkte Kommunikationsrichtung, auf wenige Schichten eingrenzen lassen. Oberhalb dieser Schichten sind sie nicht mehr sichtbar.

4.2 Schichtenmodell

Das System ist in fünf Schichten gegliedert, die auf beiden Seiten der Verbindung vorhanden sind. Abbildung 1 zeigt den Aufbau.



Abbildung 1: Schichtenmodell der Architektur. Beide Seiten sind gleich aufgebaut und verkehren jeweils nur mit der unmittelbar darunterliegenden Schicht.

Zwei Eigenschaften dieses Aufbaus sind hervorzuheben. Zum einen ist er symmetrisch. Beide Seiten besitzen dieselben Schichten mit denselben Aufgaben und unterscheiden sich nur in der Richtung, aus der ein Aufruf eintrifft. Zum anderen ist er streng geschichtet. Jede Schicht spricht ausschließlich mit der unmittelbar darunterliegenden, sodass eine Änderung innerhalb einer Schicht die übrigen nicht berührt.

Die beiden Seiten nehmen dabei feste Rollen ein. Die Steuereinheit ruft Methoden auf und stößt jede Übertragung an. Sie wird im Folgenden als Client bezeichnet. Die Anzeigeeinheit führt die Aufrufe aus und antwortet ausschließlich. Sie wird im Folgenden als Server bezeichnet. Die Rollen decken sich mit der Verteilung auf dem Bus, auf dem der Hub als Master arbeitet. Sie wechseln im Betrieb nicht, auch nicht bei Ereignissen, da auch diese vom Client abgefragt werden.

Die folgenden Abschnitte beschreiben die Schichten von unten nach oben. Für jede Schicht werden ihre Aufgabe, die nach oben angebotene Abstraktion und die Aufgaben benannt, die sie bewusst nicht übernimmt.

4.2.1 Busanbindung

Die Busanbindung ist die einzige Schicht mit Kenntnis des tatsächlichen Übertragungswegs. Ihre Aufgabe besteht darin, einen Block fester Länge zur Gegenseite zu befördern und einen ebensolchen Block als Antwort entgegenzunehmen. Sie kennt weder den Inhalt dieses Blocks noch dessen Bedeutung.

Nach oben bietet sie damit eine sehr schmale Abstraktion an, nämlich einen paketweisen, vom Client angestoßenen Austausch fester Größe. Das ist beabsichtigt, denn nur so bleibt die Schicht austauschbar. Jeder Übertragungsweg, der einen anfragegetriebenen Austausch von Blöcken fester Länge leisten kann, ist als Unterbau geeignet.

Nicht zu ihren Aufgaben gehören die Zerlegung größerer Nachrichten, die Zuordnung von Antworten zu Anfragen und jede Form der Auslegung des Inhalts. Die Paketgröße ist für diese Schicht eine gegebene Eigenschaft des Übertragungswegs und keine Größe, die sie beeinflusst.

4.2.2 Transportschicht

Die Transportschicht vermittelt zwischen Nachrichten beliebiger Länge und den kleinen Paketen des Übertragungswegs. In ihr wirkt sich damit die zentrale Einschränkung des Zielsystems aus.

Ihre erste Aufgabe ist die Serialisierung. Eine strukturierte Nachricht wird in eine Folge von Bytes überführt und auf der Gegenseite wieder ausgelesen. Ihre zweite Aufgabe ist die Rahmung. Jedes Paket erhält Steuerinformationen, mit denen die Gegenseite die ursprüngliche Nachricht wiederherstellen kann. Dazu gehören eine Kennung der Nachricht, die Position des Pakets innerhalb der Nachricht, die Angabe, ob es das letzte Paket ist, und die Anzahl der tatsächlich genutzten Bytes. Ihre dritte Aufgabe ist die Fragmentierung, also das Zerlegen einer Nachricht in eine Folge solcher Pakete und deren Zusammensetzen auf der Gegenseite.

Nach oben verbirgt diese Schicht die Paketgröße vollständig. Höhere Schichten arbeiten mit Nachrichten, deren Länge sie nicht zu beachten brauchen.

Zwei Entwurfsentscheidungen dieser Schicht sind besonders wichtig. Erstens ist die Rahmung vom Serialisierungsformat getrennt. Beide sind an je einer Stelle gebündelt, sodass sich das Format wechseln lässt, ohne die Rahmung anzutasten. Zweitens muss die Rahmung die Länge der Nutzlast ausdrücklich mitführen und darf sie nicht aus dem Inhalt ableiten, etwa durch das Entfernen von Füllbytes am Ende. Andernfalls schränkt die Rahmung die zulässigen Inhalte ein und bindet die Schicht an ein textbasiertes Format. Bei einer Nutzlast von wenigen Byte je Paket sind die Steuerinformationen außerdem ein wesentlicher Kostenfaktor der gesamten Übertragung.

4.2.3 Kommunikationsschicht

Die Kommunikationsschicht stellt den entfernten Aufruf her. Auf dem Client bildet sie aus dem Namen einer Methode, deren Argumenten und der Angabe des Wirkungsbereichs eine Nachricht, übergibt diese nach unten und nimmt anschließend das Ergebnis entgegen. Auf dem Server nimmt sie eine Nachricht entgegen, wählt den zuständigen Empfänger, ruft die benannte Methode auf und stellt deren Ergebnis zur Abholung bereit.

In dieser Schicht liegt außerdem die Behandlung des Rückkanals. Da der Server nicht von sich aus senden kann, wird das Ergebnis nicht zugestellt, sondern bereitgehalten und auf Anfrage herausgegeben. Die Kommunikationsschicht verbirgt das. Nach oben erscheint ein Aufruf als gewöhnlicher, synchroner Methodenaufruf, der zurückkehrt, sobald das Ergebnis vorliegt.

Die aufgerufenen Objekte kennt diese Schicht nicht. Sie befördert einen Aufruf zu einem benannten Empfänger, ohne zu wissen, was dieser darstellt.

4.2.4 Objektmodell

Das Objektmodell hebt den entfernten Aufruf auf die Ebene entfernter Objekte. Grafische Elemente wie Bildschirme, Beschriftungen und Schaltflächen werden als Objekte abgebildet, die auf dem Server bestehen und auf dem Client durch Stellvertreter vertreten werden.

Grundlage ist eine gemeinsame abstrakte Schnittstelle je Objekttyp, die auf beiden Seiten vorliegt. Damit ein Stellvertreter ein bestimmtes entferntes Objekt anspricht, vergibt der Server beim Erzeugen eines Objekts eine Referenz und verwaltet die Objekte in einer Registratur. Der Stellvertreter führt diese Referenz mit und übergibt sie bei jedem Aufruf. Auf diese Weise lassen sich beliebig viele Objekte erzeugen und gezielt ansprechen.

Nach oben bietet diese Schicht eine Sicht an, die sich von einer rein lokalen Programmierung nicht unterscheidet. In ihr wird damit das Ziel der Aufruftransparenz erreicht.

4.2.5 Anwendung und Anzeige

Die oberste Schicht unterscheidet sich auf beiden Seiten, da hier die eigentliche Aufgabe liegt. Auf der Steuereinheit ist es die Anwendung, die das Objektmodell nutzt und nichts von der Kommunikation weiß. Auf der Anzeigeeinheit ist es die Darstellung und die Entgegennahme von Eingaben, also die Ansteuerung der Anzeigehardware und die Erkennung von Berührungen.

Beide Seiten sind über die darunterliegenden Schichten vollständig entkoppelt. Ein Wechsel der Anzeigehardware berührt ausschließlich diese oberste Schicht auf der Anzeigeeinheit.

4.3 Rollen im Entwurf

Innerhalb der Schichten treten drei wiederkehrende Rollen auf. Sie sind als Entwurfsmuster zu verstehen und legen fest, welche Verantwortung an welcher Stelle liegt. Wie sie umgesetzt werden, bleibt offen.

4.3.1 Interceptor

Der Interceptor bildet die Schnittstelle zwischen Kommunikationsschicht und Busanbindung. Eine für beide Seiten gemeinsame Schnittstelle legt fest, wie ein Block zur Gegenseite gelangt und wie die Antwort zurückkommt. Auf dem Client verpackt der Interceptor einen Methodenaufruf, sendet dessen Pakete und holt anschließend das Ergebnis ab. Auf dem Server nimmt er die Pakete entgegen, setzt die Nachricht zusammen und stellt das Ergebnis zur Abholung bereit.

Da diese Schnittstelle ohne Bezug auf einen konkreten Übertragungsweg formuliert ist, ist der Interceptor der Ansatzpunkt für die Unabhängigkeit vom Übertragungsweg. Ein anderer Übertragungsweg wird angebunden, indem eine weitere Umsetzung dieser Rolle bereitgestellt wird. Die höheren Schichten bleiben davon unberührt.

4.3.2 Dispatcher

Der Dispatcher ordnet eine eingehende Nachricht der auszuführenden Methode zu. Jede Nachricht führt dazu die Angabe ihres Wirkungsbereichs, eine Methodenkennung und die Argumente mit sich. Anhand des Wirkungsbereichs wählt der Dispatcher den zuständigen Empfänger und ruft dort die benannte Methode auf.

Für jeden Objekttyp kann ein eigener Empfänger registriert werden. Diese Trennung entkoppelt das Routing der Nachrichten von der Umsetzung der Methoden. Das Objektmodell lässt sich dadurch schrittweise erweitern, ohne die darunterliegenden Schichten zu verändern.

4.3.3 Stellvertreter und Adapter

Stellvertreter und Adapter sind die beiden Seiten desselben Objekttyps. Der Stellvertreter implementiert die gemeinsame Schnittstelle auf dem Client und leitet jeden Methodenaufruf weiter. Der Adapter implementiert dieselbe Schnittstelle auf dem Server und enthält die eigentliche Logik, die der Dispatcher aufruft.

Dass beide dieselbe Schnittstelle erfüllen, ist die Grundlage der Aufruftransparenz. Für die Anwendung ist weder erkennbar noch relevant, ob sie mit einem Stellvertreter oder mit einer lokalen Umsetzung arbeitet.

4.4 Kommunikationsmodell

Aus Sicht der Anwendung ist die Kommunikation synchron. Ein Aufruf kehrt erst zurück, wenn das Ergebnis vorliegt. Intern folgt die Übertragung dem anfragegetriebenen Muster des Übertragungswegs. Jede Übertragung wird vom Client angestoßen, der Server antwortet nur.

Logisch werden zwei Kanäle unterschieden. Über den ersten Kanal sendet der Client die Pakete eines Aufrufs, und der Server bestätigt jedes Paket. Über den zweiten Kanal fragt der Client den Zustand und das Ergebnis ab. Er prüft zunächst, ob ein Ergebnis bereitsteht, und holt es anschließend Paket für Paket ab.

Ereignisse wie Berührungen werden nach demselben Prinzip behandelt. Der Server puffert sie, und der Client fragt sie bei Bedarf ab. Damit ist auch die Ereignisbehandlung an den Takt des Clients gebunden, was die zeitliche Auflösung begrenzt. Dieser anfragegetriebene Rückkanal ist die Antwort der Architektur auf die eingeschränkte Kommunikationsrichtung.

4.5 Datenfluss im System

Der Weg eines Aufrufs durch die Schichten lässt sich in vier Phasen gliedern. Zunächst nimmt der Stellvertreter den Methodenaufruf entgegen und übergibt ihn der Kommunikationsschicht. Diese bildet aus Methodennamen, Argumenten und Wirkungsbereich eine Nachricht. Die Transportschicht serialisiert sie, zerlegt sie in Pakete und versieht diese mit Steuerinformationen.

In der zweiten Phase überträgt der Client die Pakete. Der Server bestätigt jedes empfangene Paket und setzt die Nachricht zusammen, sobald das letzte Paket eingetroffen ist. In der dritten Phase übergibt die Kommunikationsschicht die zusammengesetzte Nachricht dem Dispatcher, der die zugehörige Methode auf dem Adapter aufruft. Das Ergebnis wird wiederum zu einer Nachricht verpackt und zur Abholung bereitgestellt.

In der letzten Phase fragt der Client ab, ob das Ergebnis bereitsteht, und holt es anschließend Paket für Paket ab. Die Kommunikationsschicht setzt die Antwort zusammen, liest die Nutzlast aus und gibt das Ergebnis an den Stellvertreter zurück, der es an die Anwendung weiterreicht.

An diesem Ablauf zeigt sich das Verhältnis von Nutzlast und Steuerverkehr. Ein einzelner Aufruf zerfällt in eine Folge von Paketen, die jeweils einzeln bestätigt werden, gefolgt von wiederholten Abfragen nach dem Ergebnis. Die Dauer eines Aufrufs hängt damit vor allem von der Anzahl der Übertragungen ab und weniger von der Geschwindigkeit des Übertragungswegs. Die Evaluation greift das auf und beziffert es.

5 Entwurf von Transport- und Kommunikationsschicht

Dieses Kapitel beschreibt den Entwurf der Transport- und der Kommunikationsschicht. Die beiden Schichten bilden die Grundlage für eine zuverlässige und effiziente Kommunikation zwischen der Anwendung und dem darunterliegenden Übertragungsweg.

Die Transportschicht, im [Open Systems Interconnection \(OSI\)](#)-Modell als Transport Layer bezeichnet, ist für die Zerlegung von Nachrichten beliebiger Länge in kleinere Pakete und deren korrekte Rekonstruktion auf der Empfängerseite verantwortlich. Dort stellt sie außerdem Funktionen wie Stauvermeidung (engl. congestion avoidance) und die Sicherstellung einer fehlerfreien Übertragung bereit [34].

Die Kommunikationsschicht setzt auf der Transportschicht auf und bildet die Schnittstelle zwischen der Anwendung und dem Transport. Sie kapselt die Details der Verbindung und stellt der Anwendung ein einfaches Kommunikationsmodell bereit. Im [OSI](#)-Modell entspricht sie am ehesten dem Session Layer, der für die Verwaltung von Kommunikationssitzungen und die Zuordnung von Anfragen und Antworten zuständig ist.

5.1 Motivation und Randbedingungen

Um die Entscheidungen in beiden Schichten nachvollziehen zu können, werden zunächst Motivation und Randbedingungen zusammengefasst. Aus den vorherigen Kapiteln geht hervor, dass die Transportschicht Nachrichten zuverlässig und effizient übertragen muss, unabhängig vom zugrunde liegenden Übertragungsweg. Dazu gehört auch die Serialisierung der Nutzlast, die in der Referenzimplementierung als Codec umgesetzt ist. Die Kommunikationsschicht muss dafür sorgen, dass Anfragen und Antworten zuverlässig zugeordnet werden, und sie stellt die Aufruftransparenz her.

5.2 Transportschicht

Wie bereits beschrieben, zerlegt die Transportschicht Nachrichten beliebiger Länge in Pakete fester Größe und setzt sie auf der Empfängerseite wieder zusammen. Ein solches Paket wird in diesem Kapitel als Block bezeichnet, sobald es um die Schnittstelle zum Übertragungsweg geht, und als Frame, sobald es die Steuerinformationen der Transportschicht trägt. Dazu muss jedes Paket Steuerinformationen tragen, wie es auch bei bekannten Protokollen wie [TCP](#) oder [User Datagram Protocol \(UDP\)](#) der Fall ist.

Im Folgenden wird beschrieben, welchen Übertragungsweg die Schicht nach unten voraussetzt. Gefordert ist lediglich, dass er Blöcke begrenzter Größe befördert. Welche [OSI](#)-Schichten dieser Weg abdeckt, ist für den Entwurf ohne Belang und je nach System verschieden. Beim hier betrachteten Bus sind es Bitübertragung und Sicherung. Anschließend werden die benötigten Steuerinformationen beschrieben, ohne feste Größen und Formate vorzugeben, damit der Entwurf auf andere Systeme übertragbar bleibt. Die konkrete Umsetzung zeigt Abschnitt 6.4.

In verteilten Systemen durchläuft eine Nachricht den Schichtenstapel zweimal, auf der Senderseite abwärts und auf der Empfängerseite aufwärts. Jede Schicht fügt auf dem Weg nach unten eigene Steuerinformationen hinzu, die auf dem Weg nach oben wieder ausgewertet und entfernt werden. Die folgenden Abschnitte betrachten deshalb stets beide Richtungen. Dieses Prinzip wird im Allgemeinen als Kapselung und Entkapselung bezeichnet [35].

5.2.1 Interceptor-Abstraktion

Die Transportschicht setzt nach unten einen Übertragungsweg voraus. Damit die Anforderungen aus Abschnitt 3.3 erfüllt werden können, darf sie über diesen Weg möglichst wenig wissen. Jede Eigenschaft, die sie voraussetzt, schließt Übertragungswege aus, die diese Eigenschaft nicht bieten. Die Schnittstelle richtet sich deshalb danach, was die betrachtete Verbindungsklasse mindestens leisten muss, und nicht nach den Fähigkeiten eines bestimmten Busses.

Da nur der Client die Übertragung anstoßen kann, lässt sich jede Interaktion auf denselben Ablauf zurückführen:

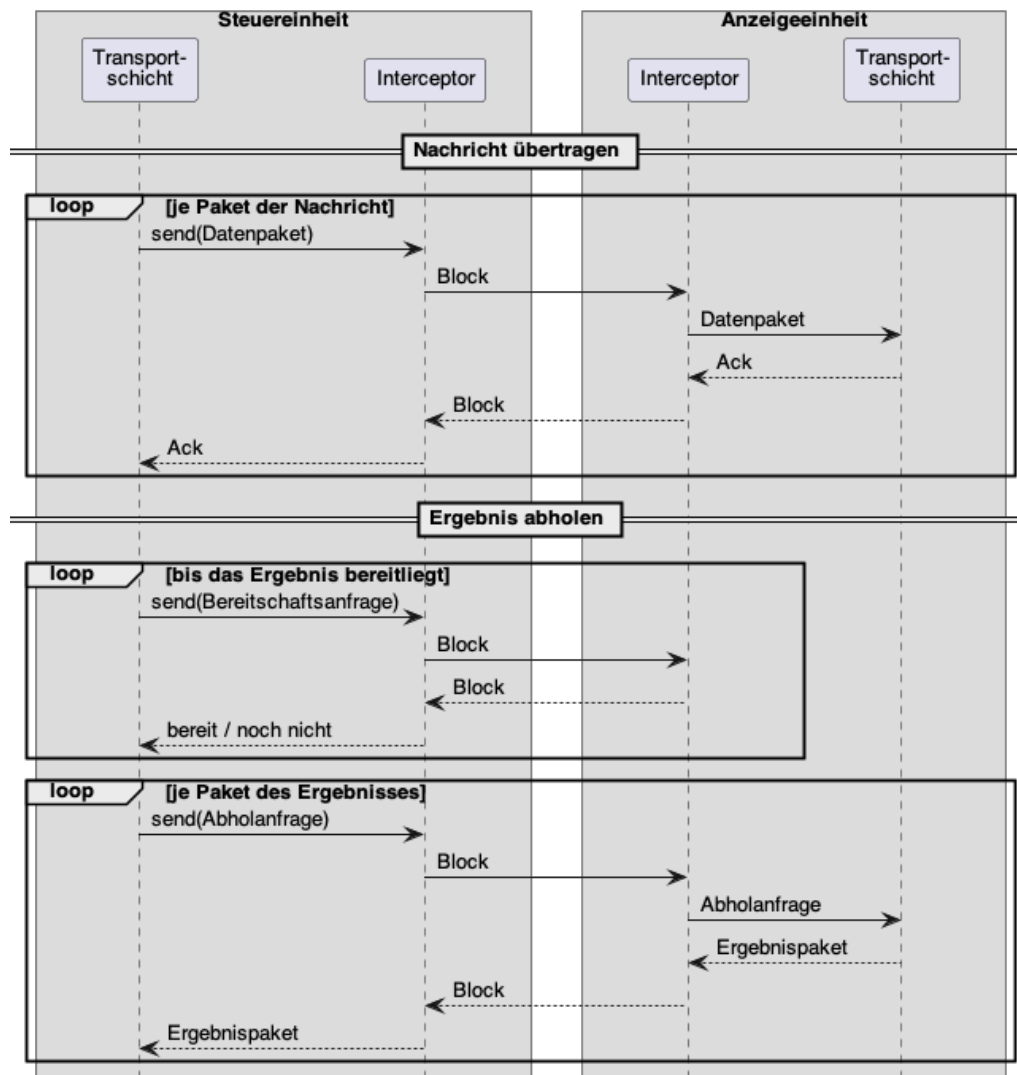


Abbildung 2: Ablauf eines Aufrufs an der Interceptor-Schnittstelle. Jede der drei Phasen besteht aus Wiederholungen derselben Operation, und jede wird vom Client angestoßen.

Die Schnittstelle bietet daher eine einzige Operation. Ein Block begrenzter Größe wird übergeben, und im selben Vorgang kommt ein Block zurück. Ob dieser Block Nutzlast, eine Bestätigung oder eine Anfrage nach dem Ergebnis enthält, spielt für die Schnittstelle keine Rolle, da sie ihn nur befördert und nicht liest. Die drei Phasen in Abbildung 2 unterscheiden sich also nur im Inhalt der Blöcke.

Eine umfangreichere Schnittstelle wäre denkbar, würde aber Übertragungswege ausschließen. Eine Zustellung ohne vorherige Anfrage setzt voraus, dass der Server von sich aus senden kann. Ein Warten auf eine Meldung des Servers setzt zusätzlich voraus, dass er dies zu einem selbst gewählten Zeitpunkt tut. Beides leistet der hier betrachtete Bus nicht, und beides ist auch für die übrigen Vertreter der in Abschnitt 3.5 beschriebenen Klasse nicht vorauszusetzen. Die Schnittstelle bildet daher die Schnittmenge dessen, was alle Übertragungswege dieser Klasse leisten können.

Beide Seiten erfüllen dieselbe Schnittstelle und unterscheiden sich allein in der Richtung, aus der ein Block eintrifft. Der Client übergibt Blöcke und erhält Antworten, der Server nimmt

Blöcke entgegen und beantwortet sie. Die Schnittstelle ist also symmetrisch, und eine Änderung des Übertragungswegs betrifft genau eine Stelle pro Seite.

Den Inhalt der Blöcke wertet die Schnittstelle nicht aus. Sie kennt weder die Zugehörigkeit eines Blocks zu einer Nachricht noch seine Position darin. Sie setzt nichts zusammen, erkennt keine Wiederholungen und ordnet keine Antworten zu. Alles, was eine Struktur voraussetzt, liegt darüber und wird in den folgenden Abschnitten beschrieben. Die Interceptor-Schnittstelle erfüllt damit die Anforderung K6 aus Abschnitt 3.3.

5.2.2 Frame-Struktur

Für die Schnittstelle aus dem vorherigen Abschnitt genügt ein Block ohne weitere Struktur. Die Transportschicht gibt den Blöcken eine Struktur, um aus einer Folge unabhängiger Blöcke eine Nachricht beliebiger Länge zu machen. Dazu benötigt jeder Block neben der Nutzlast Steuerinformationen, die unter anderem die Zugehörigkeit zu einer Nachricht und die Position innerhalb der Nachricht beschreiben. Diese Steuerinformationen werden in einem Header zusammengefasst, der jedem Block vorangestellt wird. Ein Block mit dieser Struktur wird im Folgenden als Frame bezeichnet.

Die folgenden Felder sind für die Rahmung eines Frames notwendig. Größe und Format der Felder werden dabei bewusst nicht vorgegeben, damit der Entwurf auf andere Systeme übertragbar bleibt. Eine konkrete Ausprägung zeigt Abschnitt 6.4. Abschnitt 7 vergleicht sie mit einer früheren Fassung, die dieselbe Aufgabe mit deutlich mehr Steuerinformationen löste.

Zugehörigkeit: Ohne Kennung der Nachricht lässt sich nicht entscheiden, ob ein eintreffender Block zu einer laufenden Übertragung gehört. Das ist auch bei einem rein synchronen Austausch nötig, bei dem immer nur eine Nachricht gleichzeitig unterwegs ist. Da direkt nach dem Absetzen einer Anfrage gelesen wird, hat die Gegenseite diese unter Umständen noch nicht verarbeitet. Gelesen wird dann die Antwort auf die vorangegangene Anfrage. Erst die Kennung macht solche veralteten Antworten erkennbar.

Position: Die Blöcke müssen in der richtigen Reihenfolge zusammengesetzt werden. Dazu muss die Position jedes Blocks innerhalb der Nachricht bekannt sein. Über die Position lassen sich außerdem Wiederholungen erkennen und verwerfen, die durch Übertragungsfehler oder Paketverlust entstehen.

Art: Über denselben Kanal werden sowohl Nutzlast als auch Steuerinformationen wie Bestätigungen, Anfragen nach dem Ergebnis und Fehlermeldungen übertragen. Da die Schnittstelle selbst nicht zwischen diesen Arten unterscheidet, muss der Header die Art des Blocks angeben.

Länge der Nutzlast: Der Übertragungsweg liefert Blöcke fester Größe, unabhängig davon, wie viel davon tatsächlich genutzt wird. Ohne ausdrückliche Angabe der Länge muss sie aus dem Inhalt abgeleitet werden. Warum das die Schicht an bestimmte Formate bindet, behandelt der nächste Abschnitt.

Ende der Nachricht: Der Empfänger kennt die Gesamtlänge der Nachricht nicht vorab. Daher muss das Ende der Nachricht explizit markiert werden, um zu erkennen, wann die Übertragung abgeschlossen ist. Man könnte das Ende daraus ableiten, dass der letzte Block nicht vollständig gefüllt ist. Ist die Nachrichtenlänge aber ein exaktes Vielfaches der Nutzlastgröße, ist auch der letzte Block voll und wird nicht als Ende erkannt. Der Empfänger würde dann auf einen Block warten, der nie kommt.

5.2.3 Nutzlastlänge und Formatunabhängigkeit

Neben den Steuerinformationen trägt ein Block die Nutzlast. Der Übertragungsweg liefert Blöcke fester Größe, die Länge der Nutzlast in einem Block ist aber nicht immer gleich. Eine fehlende Längenangabe ist unproblematisch, solange sich das Ende der Nutzlast aus ihrem Format ergibt. Bei textbasierten Formaten wie [JSON](#) oder [XML](#) ist das der Fall. Sie enthalten keine Nullbytes, sodass Füllbytes am Ende eines Blocks eindeutig erkannt und entfernt werden

können. Binäre Formate wie [CBOR](#) oder das in der Referenzimplementierung verwendete MessagePack besitzen diese Eigenschaft nicht.

Ohne Längenfeld ist die Transportschicht also an textbasierte Formate gebunden. Ein Wechsel des Formats oder die freie Wahl des Formats durch die Entwickler ist dann nicht möglich. Formatunabhängigkeit ist dabei keine ungewöhnliche Anforderung. Protokolle wie [HTTP](#) erlauben es beispielsweise, den Inhaltstyp der Nutzlast anzugeben, um verschiedene Formate zu unterstützen [36].

Um Formatunabhängigkeit zu erreichen, muss die Länge der Nutzlast ausdrücklich angegeben werden. Dazu nimmt der Header ein Längenfeld auf, das angibt, wie viele Bytes des Blocks tatsächlich zur Nutzlast gehören.

Die Breite dieses Feldes folgt aus der Blockgröße. Bezeichnet MTU die Größe eines Blocks und H die Größe des Headers, jeweils in Byte, so verbleibt als Nutzlast

$$P = MTU - H \quad (1)$$

Die MTU ist dabei die maximale Größe eines Blocks, die der Übertragungsweg befördern kann. Das Längenfeld muss jeden Wert von 0 bis P ausdrücken können. Seine Breite b in Bit beträgt damit

$$b = \lceil \log_2(P + 1) \rceil \quad (2)$$

Da das Längenfeld selbst Teil des Headers ist, hängen Formel 1 und Formel 2 voneinander ab: H bestimmt P , P bestimmt b , und b geht wieder in H ein. Aufgelöst wird das, indem der kleinste Header gewählt wird, für den beide Gleichungen zugleich gelten. Abschnitt 6.4 führt diese Rechnung für die konkrete Blockgröße aus.

Das Längenfeld ist damit die Voraussetzung für Formatunabhängigkeit.

5.2.4 Kosten der Rahmung

Jedes Byte des Headers ist Overhead und steht nicht für die Nutzlast zur Verfügung. Bei den hier betrachteten Blockgrößen bestimmt dieser Overhead den Aufwand der Übertragung maßgeblich.

Deutlich wird das, wenn Blöcke statt Bytes gezählt werden. Eine Nachricht der Länge L zerfällt in

$$N = \left\lceil \frac{L}{P} \right\rceil \quad (3)$$

Blöcke. Übertragen werden damit $N \cdot MTU$ Bytes, unabhängig davon, wie viel davon genutzt wird. Der letzte Block ist in der Regel nur teilweise gefüllt und wird trotzdem mit der vollen Blockgröße übertragen. Als Maß dient die Nutzlasteffizienz, also das Verhältnis von tatsächlich genutzten zu übertragenen Bytes:

$$\eta = \frac{L}{N \cdot MTU} \quad (4)$$

Daran lässt sich ablesen, wie stark Headergröße und Blockgröße die Effizienz beeinflussen. Wird der Header bei gleicher Blockgröße um ein Byte verkleinert, wächst P um eins. Nach Formel 3 kann das N um einen ganzen Block verringern. Eingespart wird dann eine vollständige Übertragung und damit ein Round Trip.

Neben Blockgröße und Headerbreite beeinflusst auch die Länge der Nutzlast L die Effizienz. Sie hängt selbst bei gleichem Inhalt vom gewählten Serialisierungsformat ab. Wie sich zwei verschiedene Formate auswirken, vergleicht Abschnitt 7. Die Nutzlasteffizienz hängt also von drei Faktoren ab, nämlich von Blockgröße, Headerbreite und Länge der Nutzlast.

Headerbreite und Nutzlastlänge wirken über dieselbe Aufrundung, aber auf unterschiedliche Weise. Eine kürzere Nutzlast senkt N nur dann, wenn sie eine Blockgrenze unterschreitet. Ein kleinerer Header verschiebt dagegen alle Blockgrenzen zugleich. Bei kleiner MTU ist deshalb zu erwarten, dass die Rahmung stärker auf die Anzahl der Übertragungen wirkt als die Wahl des

Serialisierungsformats, obwohl das Format die Nutzlast direkter verkleinert. Abschnitt 7 prüft diese Erwartung.

5.2.5 Fragmentierung und Rekonstruktion

Die Zerlegung einer Nachricht folgt unmittelbar aus den bisherigen Festlegungen. Die serialisierte Nutzlast wird in Abschnitte von höchstens P Byte geteilt, jeder Abschnitt erhält die Kennung der Nachricht und seine Position, und der letzte wird als solcher markiert. Der letzte Abschnitt ist dabei in der Regel kürzer als P , die tatsächliche Länge trägt das Längenfeld.

Wichtig ist, dass diese Zuordnung eindeutig ist. Zu einer Nachricht und einer Position gehört immer derselbe Block. Die Zerlegung ist damit wiederholbar und nicht an einen Fortschritt gebunden. Sie ist also idempotent, und die Wiederholung einer Übertragung verändert das Ergebnis nicht.

Wann ist eine Nachricht vollständig? Die Gesamtlänge einer Nachricht ist dem Empfänger nicht bekannt, er kann die Vollständigkeit also nicht durch einen Vergleich mit ihr feststellen. Erkennbar ist sie nur an den eingetroffenen Blöcken. Trägt der Block an Position n die Endmarkierung, besteht die Nachricht aus $n + 1$ Blöcken. Vollständig ist sie, sobald jede Position von 0 bis n mindestens einmal eingetroffen ist.

Die Steuerinformationen haben dabei verschiedene Aufgaben. Endmarkierung und Position bestimmen, wie viele Blöcke zur Nachricht gehören. Das Längenfeld bestimmt, wie viele Bytes eines Blocks dazugehören. Die Prüfung auf Vollständigkeit kommt damit ohne Kenntnis des Inhalts aus.

Was passiert bei Übertragungswiederholungen? Ein Block wird erneut übertragen, wenn die zugehörige Bestätigung ausbleibt. Das kann daran liegen, dass er den Empfänger nicht erreicht hat, dass ihn eine Prüfsumme des Übertragungswegs als beschädigt verworfen hat oder dass die Bestätigung selbst verloren ging. Für den Sender sind diese Fälle nicht unterscheidbar, weshalb er in allen gleich reagiert und den Block erneut sendet. Die Mechanismen dafür werden in Abschnitt 5.2.7 beschrieben.

Da die Zerlegung eindeutig ist, hat der wiederholte Block denselben Inhalt wie der ursprüngliche. Das allein hilft dem Empfänger aber nicht. Hängt er den Block ein zweites Mal an, entsteht eine falsche Nachricht. Die Rekonstruktion muss Wiederholungen deshalb selbst abfangen und Blöcke verwerfen, deren Position bereits vorliegt. Erst dadurch ist auch sie idempotent.

Was passiert bei einer abgebrochenen Übertragung? Bricht eine Übertragung ab, bevor die Nachricht vollständig ist, bleiben die bereits eingetroffenen Blöcke liegen. Der Empfänger kann nicht entscheiden, ob die Übertragung noch fortgesetzt wird, da er weder die Gesamtzahl der Blöcke noch einen Zeitpunkt kennt, zu dem er aufgeben dürfte. Erkennbar wird der Abbruch erst daran, dass eine neue Übertragung beginnt, also ein Block an Position 0 eintrifft. Die Position ist dafür verlässlicher als die Kennung, da sich die Kennung nach einer festen Anzahl von Nachrichten wiederholt.

Für den Umgang mit den liegengebliebenen Blöcken bestehen zwei Möglichkeiten:

- Die unvollständige Nachricht wird verworfen, sobald eine neue beginnt. Das ist der einfachere Weg, setzt aber voraus, dass nie mehr als eine Nachricht gleichzeitig übertragen wird.
- Die unvollständige Nachricht bleibt erhalten und kann später fortgesetzt werden. Das erlaubt mehrere gleichzeitige Übertragungen, verlangt aber einen Puffer je Kennung, eine Regel, wann eine Nachricht endgültig aufgegeben wird, und eine Laufzeitumgebung, die mehrere Übertragungen nebenläufig verwalten kann.

Der Entwurf wählt die erste Möglichkeit. Die betrachtete Verbindungsklasse ist anfragegetrieben, sodass der Client ohnehin nur eine Nachricht zugleich senden kann. Hinzu kommt, dass die in Abschnitt 3.4 beschriebene Laufzeitumgebung die für die zweite Möglichkeit nötige Nebenläufigkeit nicht bereitstellt.

Wie viel Speicher benötigt die Rekonstruktion? Der Empfänger muss alle Blöcke einer Nachricht halten, bis sie vollständig ist. Anforderung K5 verlangt dafür eine zur Entwurfszeit bekannte Obergrenze. Sie ergibt sich aus den bisherigen Festlegungen. Ist b_{pos} die Breite des Positionsfelds in Bit, lassen sich $2^{b_{\text{pos}}}$ Positionen unterscheiden, und die längste übertragbare Nachricht umfasst höchstens

$$L_{\text{max}} = 2^{b_{\text{pos}}} \cdot P \quad (5)$$

Bytes. Dieser Wert ist zugleich die Obergrenze des Puffers, den die Gegenseite bereithalten muss.

5.2.6 Serialisierung

Die Serialisierung überführt einen strukturierten Wert in eine Bytefolge und liest ihn auf der Gegenseite wieder aus. Sie liegt damit zwischen der Kommunikationsschicht, die mit Werten arbeitet, und der Fragmentierung, die mit Bytes arbeitet. Aus Abschnitt 5.2.3 folgt dabei eine Reihenfolge. Das Format kann erst dann frei gewählt werden, wenn die Rahmung die Länge der Nutzlast ausdrücklich mitführt.

Für die Transportschicht ist die Nutzlast ohne Struktur. Sie nimmt eine Bytefolge entgegen, zerlegt sie und setzt sie auf der Gegenseite wieder zusammen. Welche Werte darin vorkommen und wie sie angeordnet sind, wertet sie nicht aus. Das Format ist damit austauschbar, ohne dass die Rahmung geändert werden muss.

Die Wahl des Formats ist an drei Bedingungen gebunden, die sich aus den bisherigen Abschnitten ergeben.

Erstens muss das Format kompakt sein. Nach Formel 3 bestimmt die Länge der Nutzdaten unmittelbar die Anzahl der Blöcke, in die eine Nachricht zerfällt, und damit die Anzahl der Übertragungen.

Zweitens muss es ohne Schema auskommen. Welche Objekte und Methoden es gibt, steht nicht vorab fest und ändert sich während der Entwicklung. Ein schemagebundenes Format würde einen Übersetzungsschritt einführen und die Kommunikationsschicht an diesen Schritt binden.

Drittens muss der Decoder klein sein. Er läuft auf beiden Seiten der Verbindung, und beide sind Mikrocontroller. Anforderung K5 gilt hier also auch für den Umfang der eingesetzten Bibliothek und nicht nur für die Puffer.

Welches Format diese Bedingungen erfüllt, zeigt Abschnitt 6. Wie stark sich die Wahl auf die Anzahl der Übertragungen auswirkt, misst Abschnitt 7.

5.2.7 Zuverlässigkeitsmechanismen

Die bisherigen Abschnitte setzen voraus, dass ein Block die Gegenseite erreicht. Garantiert ist das nicht. Ein Block kann auf dem Weg verloren gehen oder vom Übertragungsweg als beschädigt verworfen werden. Außerdem kann die Bestätigung ausbleiben, obwohl der Block angekommen ist.

Für diese Fälle wird Stop-and-Wait [Automatic Repeat reQuest \(ARQ\)](#) verwendet. Dabei wird ein Block gesendet und auf dessen Bestätigung gewartet. Bleibt sie aus, wird der Block erneut gesendet. Verfahren mit Sendefenster erlauben mehrere unbestätigte Blöcke gleichzeitig und erreichen dadurch einen höheren Durchsatz. Sie sind weit verbreitet und werden beispielsweise in Mobilfunkstandards und in [TCP](#) eingesetzt. Sie setzen jedoch voraus, dass mehrere Blöcke gleichzeitig unterwegs sein können und die Gegenseite Bestätigungen ohne eigene Anfrage zurücksenden kann. In der betrachteten Verbindungsklasse besteht jede Übertragung aus Anfrage und Antwort, sodass ohnehin nie mehr als ein Block unterwegs ist. Stop-and-Wait bildet damit genau das ab, was die Verbindung zulässt.

Die drei genannten Fehlerfälle sind für den Sender nicht unterscheidbar. Er sieht in allen Fällen dasselbe, nämlich eine ausbleibende Bestätigung. Eine Unterscheidung würde auch nichts

bringen, da die Reaktion immer dieselbe ist. Der Empfänger erkennt die Wiederholung an der Position und verwirft sie, wie in Abschnitt 5.2.5 beschrieben.

Wiederholungen brauchen eine Obergrenze. Ohne sie wartet der Client endlos, sobald der Server dauerhaft nicht antwortet. Aus einem Fehler auf der einen Seite wird dann ein Stillstand der Anwendung auf der anderen. Mit einer Obergrenze scheitert der Aufruf nach einer bekannten Zeit und die Anwendung behält die Kontrolle. Das ist zugleich die Voraussetzung dafür, dass K4 geprüft werden kann, denn eine Antwortzeit lässt sich nur angeben, wenn sie nach oben begrenzt ist.

Dieselbe Überlegung gilt für das Abholen des Ergebnisses. Der Client fragt so lange nach, bis ein Ergebnis vorliegt, und braucht auch dafür eine Obergrenze. Für den Server folgt daraus eine Zusicherung. Jeder Aufruf muss ein Ergebnis oder eine Fehlermeldung hinterlassen, auch wenn die Ausführung selbst fehlschlägt. Bleibt beides aus, wartet der Client auf etwas, das nie eintrifft. Die Fehlerbehandlung ist damit Teil des Protokolls und wird nicht der Umsetzung überlassen.

Drei Aufgaben übernimmt die Schicht ausdrücklich nicht. Sie korrigiert keine Fehler, sondern verlässt sich auf die Prüfsumme des Übertragungswegs und wiederholt den betroffenen Block. Sie sichert die Reihenfolge nicht gesondert ab, da der Kanal sequenziell arbeitet. Das Positionsfeld macht die Rekonstruktion trotzdem von der Reihenfolge unabhängig. Außerdem betreibt sie keine Stauvermeidung. Auf einer Punkt-zu-Punkt-Verbindung mit einem Master und einer einzigen Nachricht gleichzeitig kann kein Stau entstehen.

5.3 Kommunikationsschicht

Mit der Transportschicht lassen sich Nachrichten beliebiger Länge übertragen. Für eine Anwendung ist das noch keine geeignete Schnittstelle, da sie mit Methodenaufrufen arbeitet. Diese Lücke schließt die Kommunikationsschicht. Sie bildet einen Aufruf auf eine Nachricht ab, befördert ihn über die Transportschicht und gibt das Ergebnis an die aufrufende Stelle zurück.

Von der Transportschicht kennt sie nur zwei Vorgänge. Eine Nachricht wird gesendet, und eine Nachricht kommt zurück. In wie viele Blöcke die Nachricht dabei zerfällt, wie oft ein Block wiederholt wird und in welchem Format die Nutzlast vorliegt, bleibt verborgen.

Die Schicht stellt einen [RPC](#) her, wie er in Abschnitt 2 eingeführt wurde. Die Anwendung ruft eine Methode auf, die auf einem anderen Gerät ausgeführt wird, und erhält deren Rückgabewert. Dass dabei eine Übertragung stattgefunden hat, ist an der Aufrufstelle nicht erkennbar. Dieses Verhalten wird als Aufrufstransparenz bezeichnet.

Die dort betrachteten Rahmenwerke setzen dafür Eigenschaften voraus, die diese Verbindung nicht bietet. Sie erwarten einen Transport, der Nachrichten beliebiger Länge zuverlässig befördert, und einen Rückkanal, über den der Server sein Ergebnis von sich aus zustellt. Beides ist hier nicht gegeben. Die folgenden Abschnitte entwickeln deshalb einen eigenen Aufrufmechanismus, der mit den Mitteln der Transportschicht auskommt.

Dabei ist zu klären, welche Angaben eine Aufrufnachricht mitführen muss, wie eine Antwort ihrer Anfrage zugeordnet wird, wie der Server den zuständigen Empfänger auswählt und auf welchem Weg ein Ergebnis zurückgelangt.

5.3.1 Aufbau einer Aufrufnachricht

Ein [RPC](#) muss so viel mitführen, dass der Server den Aufruf ohne weitere Rückfrage durchführen kann. Dafür sind drei Angaben nötig. Die Methodenkennung legt fest, was ausgeführt wird. Die Argumente liefern die Werte, mit denen es ausgeführt wird. Der Wirkungsbereich bestimmt, welcher Empfänger die Methode ausführt. Wie das funktioniert, beschreibt der übernächste Abschnitt.

Angaben über Typen oder Signaturen werden nicht mitgeführt. Nach Abschnitt 5.2.6 kommt das Format ohne Schema aus, und übertragen werden nur Werte, keine Beschreibungen

von Werten. Ob eine Methode existiert oder bestimmte Argumente erwartet, stellt sich erst auf dem Server heraus. Die Prüfung findet also zur Laufzeit statt.

Das Ergebnis nimmt denselben Weg zurück. Es ist ebenfalls eine Nachricht und trägt entweder den Rückgabewert oder eine Fehlermeldung. Welcher der beiden Fälle vorliegt, ist am Schlüssel zu erkennen, unter dem der Inhalt steht. Das folgt aus der Zusicherung in Abschnitt 5.2.7, denn der Client wartet auf genau eine Antwort und muss auch im Fehlerfall eine erhalten.

Diese Angaben werden mit jedem Aufruf übertragen, und nach Formel 3 wirkt sich ihre Länge direkt auf die Anzahl der Blöcke aus. Zwei Festlegungen halten sie deshalb kurz. Die Argumente werden in der Reihenfolge der Parameter übertragen, ohne ihre Namen. Die Methode wird über eine Kennung fester Länge bezeichnet, die beide Seiten auf dieselbe Weise aus ihrem Namen berechnen, sodass keine Tabelle zwischen Namen und Kennungen gepflegt werden muss.

Beide Festlegungen setzen voraus, dass Client und Server dieselbe Schnittstelle kennen. Sie erreichen damit dasselbe wie ein schemagebundenes Format, das Bezeichner einmal vorab vereinbart und in der Nachricht nur noch Werte überträgt. Als Vereinbarung dient hier die gemeinsame Schnittstelle der Objekttypen aus Abschnitt 4. Es entsteht kein eigener Übersetzungsschritt, und das Serialisierungsformat bleibt schemafrei.

Fehler in dieser Vereinbarung fallen unterschiedlich auf. Eine unbekannte Methodenkennung erkennt der Server und antwortet mit einer Fehlermeldung. Eine abweichende Reihenfolge der Argumente fällt dagegen nur auf, wenn sich dabei ihre Anzahl ändert. Vertauschte Werte gleicher Art werden ohne Fehler ausgeführt.

5.3.2 Zuordnung von Anfrage und Antwort

Jede Antwort muss der Anfrage zugeordnet werden, zu der sie gehört. Der Frame führt nach Abschnitt 5.2.2 bereits eine Kennung je Nachricht mit, und ein Aufruf entspricht genau einer Nachricht. Diese Kennung identifiziert also zugleich den Aufruf, und eine zweite Angabe in der Nutzlast ist nicht nötig.

Die Kennung stammt damit aus dem Header, und ihr Wertebereich folgt aus dessen Breite. Sie wiederholt sich nach einer festen Anzahl von Aufrufen. Das ist zulässig, solange zwischen zwei gleichen Kennungen kein älteres Ergebnis mehr unterwegs sein kann. Eine eigene Kennung der Kommunikationsschicht wäre von der Headerbreite unabhängig, müsste aber in jeder Nachricht zusätzlich übertragen werden und würde nach Formel 3 die Anzahl der Übertragungen erhöhen. Die Übernahme der Kennung aus dem Header ist also eine Abwägung zwischen doppelten Steuerinformationen und einer strengen Trennung der Schichten.

5.3.3 Verteilung eingehender Aufrufe

Der Server erhält eine Nachricht mit Methodenkennung, Argumenten und Wirkungsbereich und muss daraus bestimmen, wer den Aufruf ausführt. Die Adressierung ist zweistufig und mit dem Routing in [Representational State Transfer \(REST\)](#)-Frameworks vergleichbar. Dort ist ein Controller für einen Ressourcentyp zuständig, eine Methode darin für die Operation, und die Kennung der einzelnen Ressource kommt als Parameter an. Hier benennt der Wirkungsbereich die Art des Empfängers, der Methodenname die Operation, und die Referenz des einzelnen Objekts reist als Argument mit.

Der Unterschied liegt im Satz der Operationen. [REST](#) beschränkt ihn auf wenige festgelegte Verben, während hier jede Methode des Objektmodells aufrufbar ist. Die Referenz ist außerdem kein Pfad, sondern eine Kennung, die der Server beim Erzeugen des Objekts vergibt. Wie sie gebildet wird, gehört zum Objektmodell und wird in Abschnitt 6.6 gezeigt.

Die zweistufige Adressierung hat einen weiteren Vorteil. Die Kennungen der Methoden müssen nur innerhalb eines Wirkungsbereichs eindeutig sein. Gleichnamige Methoden können auf verschiedenen Arten von Empfängern nebeneinander bestehen, und eine zufällige Überein-

stimmung zweier Kennungen stört nur, wenn sie im selben Wirkungsbereich auftritt. Der Server prüft das beim Registrieren eines Empfängers und nicht erst beim Aufruf.

Verteilung und Ausführung bleiben dabei getrennt. Die Verteilung wählt anhand des Wirkungsbereichs den Empfänger aus, die Ausführung liegt beim Empfänger selbst. Ein weiterer Empfänger kann registriert werden, ohne die Verteilung oder die darunterliegenden Schichten zu ändern. Dadurch lässt sich das Objektmodell erweitern.

Findet sich für einen Wirkungsbereich kein Empfänger, muss der Aufruf mit einer Fehlermeldung beantwortet werden. Nach Abschnitt 5.2.7 wartet der Client auf genau eine Antwort, und ein stillschweigendes Verwerfen würde ihn bis zur Obergrenze warten lassen.

5.3.4 Anfragegetriebener Rückkanal

Aufgrund der eingeschränkten Kommunikationsrichtung aus Abschnitt 3.4 kann der Server keine Übertragung anstoßen. Ein Ergebnis kann er deshalb nicht zustellen, sondern nur bereithalten, bis es abgeholt wird. Hinzu kommt, dass die Ausführung noch nicht abgeschlossen ist, wenn der letzte Block des Aufrufs bestätigt wurde. Der Client muss also zunächst feststellen, ob ein Ergebnis vorliegt, und es anschließend abholen.

Ereignisse folgen demselben Muster, unterscheiden sich aber in einem Punkt. Ein Ergebnis erwartet der Client, weil er den zugehörigen Aufruf abgesetzt hat. Ein Ereignis erwartet er dagegen nicht. Er muss also regelmäßig nachfragen, ohne zu wissen, ob es etwas abzuholen gibt.

Die zeitliche Auflösung hängt damit vom Abfragetakt ab. Ein Ereignis, das zwischen zwei Abfragen auftritt, wird erst bei der nächsten bemerkt. Häufigeres Abfragen verkürzt diese Verzögerung, kostet aber je Abfrage eine Übertragung und damit Zeit, die für Aufrufe fehlt. Selteneres Abfragen spart Übertragungen, verlängert aber die Verzögerung. Beides lässt sich nicht gleichzeitig verbessern.

Zwischen zwei Abfragen können mehrere Ereignisse auftreten. Der Server muss sie puffern, und der Puffer ist nach K5 begrenzt. Läuft er über, gehen Ereignisse verloren. Für diesen Fall muss ein Verhalten festgelegt sein, da ein stilles Verwerfen für die Anwendung nicht erkennbar wäre.

Am anfragegetriebenen Rückkanal wirkt sich die eingeschränkte Kommunikationsrichtung direkt auf die Anwendung aus. Ob die erreichbare Antwortzeit die Anforderung K4 einhält, prüft Abschnitt 7. Eine Möglichkeit, diesen Weg für reine Darstellungsreaktionen zu umgehen, zeigt Abschnitt 6.7.

6 Referenzimplementierung

6.1 Laufzeitumgebung

Tabelle III ordnet die Module der Referenzimplementierung den Schichten aus Abschnitt 4 zu.

Tabelle III: Zuordnung der Module zu den Schichten aus Abschnitt 4

Schicht	Module
Anwendung und Anzeige	hub/main.py , display/server/driver/
Objektmodell	display/protocol/ , display/client/ mit remote.py , display/server/ , middleware/rpc/registry.py
Kommunikations-schicht	middleware/rpc/dispatcher.py , middleware/rpc/names.py , middleware/errors.py , Aufrufteil von transport/pupremote/client.py und server.py
Transportschicht	middleware/frame.py , message.py , codec.py , queue.py , constants.py , Übertragungsteil von client.py und server.py
Busanbindung	transport/base.py , transport/pupremote/binding.py , vendor/pupremote/

6.2 Hardwareaufbau

6.2.1 Aufbau und Komponenten

6.2.2 Signalzuordnung und Aufbau der Adapterplatine

6.3 Busanbindung

Die Busanbindung wird nicht entworfen, sondern vorgefunden. Beide Seiten verwenden PUPRemote [27], auf dem Hub als PUPRemoteHub und auf dem ESP32 als PUPRemoteSensor. Die Bibliothek setzt das LPF2-Protokoll um, über das der Hub mit angeschlossenen Sensoren kommuniziert.

Gegenüber dem Hub gibt sich der ESP32 als Ultraschallsensor aus, da der Hub nur Geräte mit bekannter Kennung annimmt. Beim Verbindungsaufbau handeln beide Seiten zunächst bei 2400 Baud die Eigenschaften des Geräts aus und wechseln anschließend auf 115200 Baud. Danach überwacht die Bibliothek die Verbindung selbstständig und baut sie nach einem Abbruch neu auf. Zusätzlich fordert der ESP32 die 8-V-Versorgung am Port an, da das Display mehr Strom benötigt, als die Logikversorgung liefert.

PUPRemote bildet benannte Kommandos auf die Modi des Busses ab und bringt dafür eine eigene Aufrufsemantik mit festen Formaten je Kommando mit. Diese wird hier nicht genutzt. Registriert ist ein einziges Kommando `xfer`, dessen Format in beide Richtungen ein Block von 16 Byte ist. Für PUPRemote hat dieser Block keine Struktur. Alles Weitere liegt in den Schichten darüber.

```
COMMAND = "xfer"

def register_interceptor(self):
    self.remote.add_command(
        self.COMMAND,
        from_hub_fmt=f"{FRAME_SIZE}s",
        to_hub_fmt=f"{FRAME_SIZE}s",
    )
```

Listing 1: Registrierung des einzigen Kommandos

Ein Austausch im Sinne von Abschnitt 5.2.1 ist auf dem Hub ein Aufruf von `remote.call`. Dabei wird der Block in den Modus des Kommandos geschrieben und anschließend der Inhalt desselben Modus zurückgelesen. Zwischen beidem wartet die Bibliothek nicht, sofern keine Wartezeit angegeben ist. Hat der ESP32 den Block bis zum Lesen noch nicht verarbeitet, liefert der Lesevorgang die vorige Antwort. Solche veralteten Antworten erkennen die darüberliegenden Schichten an Kennung, Position und Art des Frames, wie in Abschnitt 5.2.7 beschrieben.

Auf dem ESP32 ruft ein Task der Ereignisschleife etwa jede Millisekunde `process` auf, wobei je Durchlauf höchstens ein Kommando bearbeitet wird. Für jedes eintreffende Kommando ruft PUPRemote den registrierten Rückruf auf. Die Bibliothek findet ihn, indem sie den Kommandonamen im Namensraum des Hauptprogramms auswertet. Das Hauptprogramm muss den Rückruf deshalb unter dem Namen `xfer` bereitstellen.

Zwei Grenzen der Busanbindung wirken sich auf den gesamten Entwurf aus. Die Blockgröße ist mit der Firmware des Hubs auf 16 Byte begrenzt, darüber treten Prüfsummenfehler auf. Außerdem dürfen Kommandonamen bei angeforderter 8-V-Versorgung höchstens fünf Zeichen lang sein. Oberhalb dieser Schicht ist davon nur noch die Blockgröße sichtbar, und zwar als Zahl, aus der Abschnitt 6.4 die Nutzlast ableitet.

6.4 Transportschicht

6.4.1 Frame

Die Blockgröße ist durch die Busanbindung vorgegeben. Die Firmware des Hubs begrenzt einen Modus auf 16 Byte. Damit steht die in Abschnitt 5.2.3 eingeführte `MTU` fest, und alle weiteren Größen leiten sich daraus ab.

Der Header belegt drei Byte.

Tabelle IV: Aufbau des Frame-Headers

Byte	Bits	Inhalt
0	8	Kennung der Nachricht
1	8	Position innerhalb der Nachricht
2	4 obere	Art des Frames
2	4 untere	Länge der Nutzlast in Byte

Für die Nutzlast verbleiben damit 13 Byte je Frame.

Diese Aufteilung erfüllt die Bedingungen aus Abschnitt 5.2.3. Bei einem Header von drei Byte ergibt Formel 1 eine Nutzlast von 13 Byte, und Formel 2 verlangt dafür ein vier Bit breites Längenfeld. Die obere Hälfte des dritten Bytes enthält die Art des Frames, die untere Hälfte die Länge der Nutzlast. Das Längenfeld vergrößert den Header damit nicht. Ein Header von zwei Byte wäre nur möglich, wenn Kennung oder Position schmaler ausfielen. Das würde die Anzahl unterscheidbarer Nachrichten oder die maximale Nachrichtenlänge verringern.

Listing 2 zeigt die Umwandlung von Bytes in einen Frame. Dabei wird auch eine zu kurze Eingabe abgefangen und das dritte Byte in Art und Länge aufgeteilt.

```

@classmethod
def from_bytes(cls, data):
    if data is None or len(data) < cls.HEADER_SIZE:
        return cls.create_error(payload=b"short")

    g_id, frame_nr, b2 = unpack(cls._header_fmt, bytes(data[:cls.HEADER_SIZE]))
    length = b2 & 0x0F
    start = cls.HEADER_SIZE

    return cls(g_id, frame_nr, b2 >> 4, data[start:start + length])

```

Listing 2: Funktion zur Erzeugung eines Frames aus Bytes

Gültig bleibt die Aufteilung, solange die Nutzlast 15 Byte nicht überschreitet, da das Längenfeld keine größeren Werte darstellen kann. Bei einem Header von drei Byte entspricht das einer Blockgröße von höchstens 18 Byte. Größere Blöcke würden ein Längenfeld von einem vollen Byte und damit einen Header von vier Byte erfordern. Listing 3 zeigt die umgekehrte Umwandlung eines Frames in Bytes, bei der eine zu lange Nutzlast abgewiesen wird.

```

def to_bytes(self):
    n = len(self.payload)
    if n > MAX_PAYLOAD:
        raise ValueError("payload grew beyond {} bytes: {}".format(MAX_PAYLOAD, n))

    b2 = (self.opcode << 4) | n

    return pack(self._header_fmt, self.g_id, self.frame_nr, b2) + self.payload

```

Listing 3: Funktion zur Umwandlung eines Frames in Bytes

Die Position hat durch ihre Breite von acht Bit einen Wertebereich von 0 bis 255. Der Wert `0xFF` ist für Frames reserviert, die sich auf keine Position innerhalb einer Nachricht beziehen. Für Nutzlastframes stehen damit 255 Werte zur Verfügung, was die Länge einer Nachricht auf $13 \cdot 255 = 3315$ Byte begrenzt. Dieser Wert ist zugleich die nach K5 geforderte Obergrenze des Puffers.

Für die Art des Frames sind von 16 darstellbaren Werten acht belegt (siehe Tabelle V). Acht Werte kämen gerade noch mit drei Bit aus, allerdings bliebe dann kein Wert für eine weitere Art frei. Das eingesparte Bit würde außerdem nichts bringen. Im Längenfeld könnte es Werte bis 31 darstellen, obwohl die Nutzlast 13 Byte nie überschreitet. In der Position würde es über die Bytegrenze hinausreichen und Bitoperationen über zwei Byte erfordern. Die Breite von vier Bit ist daher eine pragmatische Entscheidung.

Tabelle V: Werte für die Art des Frames und ihre Bedeutung

Wert	Name	Bedeutung
0x0	DATA	Nutzlastframe, weitere folgen
0x1	DATA_LAST	Nutzlastframe, letzter der Nachricht
0x2	ACK	Bestätigung eines empfangenen Frames
0x3	NACK	Ergebnis noch nicht bereit oder nichts weiter vorhanden
0x4	ERR	Fehlermeldung, kann eine kurze Beschreibung tragen
0x5	READY	Anfrage, ob ein Ergebnis bereitliegt
0x6	NEXT	Anfrage nach dem Ergebnisframe an der angegebenen Position
0x7	DONE	Antwort auf READY, das Ergebnis liegt zur Abholung bereit

6.4.2 Message

Ein `Message`-Objekt enthält die Kennung und die serialisierte Nutzlast einer Nachricht. Es dient beiden Seiten als Container für die Nachricht, die über den Bus übertragen wird. Beim Senden

wird die Nutzlast gesetzt, und das Objekt liefert daraus die Frames. Beim Empfangen werden Frames gesammelt, und daraus wird die Nutzlast rekonstruiert.

Die Zerlegung ist als Funktion über Position und Nutzlast umgesetzt. In einer früheren Version lieferte ein Generator die Frames nacheinander. Diese Variante wurde verworfen, da sie das Wiederholen einzelner Frames erschwerte.

Listing 4 berechnet den zugehörigen Abschnitt direkt aus dem Index, ohne einen Fortschritt mitzuführen. Dieselbe Position liefert damit immer denselben Frame, wie es Abschnitt 5.2.5 fordert. Der Rückkanal ist darauf angewiesen, da er Ergebnisframes einzeln über ihre Position anfordert und dieselbe Position mehrfach anfragen kann.

```
def frame_at(self, index: int):
    if index < 0:
        return None

    start = index * PAYLOAD_SIZE
    if start >= len(self.payload) and not (index == 0 and not self.payload):
        return None

    chunk = self.payload[start:start + PAYLOAD_SIZE]
    last = start + PAYLOAD_SIZE >= len(self.payload)

    return Frame(
        g_id=self.g_id,
        frame_nr=index % 256,
        opcode=OP_DATA_LAST if last else OP_DATA,
        payload=chunk,
    )
```

Listing 4: Funktion zur Erzeugung von Frames aus einer Nachricht

Für das Einsammeln eingehender Frames gibt es zwei Wege, die sich in Speicherbedarf und Zusicherung unterscheiden. `collect_frame` behält jeden Frame, bis die Nachricht vollständig ist, und sortiert die Frames nach ihrer Position. Diese Variante ist von der Reihenfolge der Ankunft unabhängig, hält aber alle Frames einer Nachricht gleichzeitig im Speicher. `append_frame` hängt die Nutzlast direkt an und verwirft den Frame. Das spart Speicher, setzt aber die richtige Reihenfolge voraus. `get_payload` liefert in beiden Fällen die vollständige Nutzlast.

Der Rückkanal verwendet den sammelnden Weg, der Hinweg den anhängenden. Die in Abschnitt 5.2.5 beschriebene Unabhängigkeit von der Reihenfolge ist damit nur auf dem Rückkanal umgesetzt. Für den hier betrachteten Bus genügt das, da er sequenziell arbeitet. Bei einem Übertragungsweg, der umsortiert, müsste auch der Hinweg auf den sammelnden Weg wechseln.

6.4.3 Serialisierung

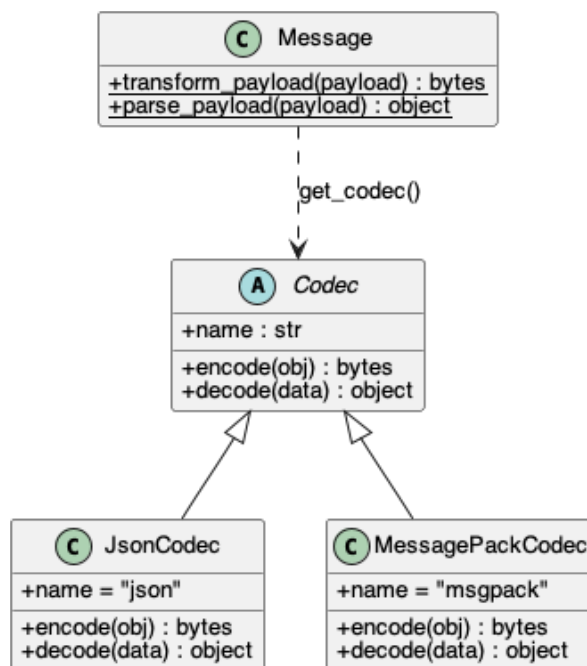


Abbildung 3: Klassendiagramm des Codecs

Die Wahl des Formats ist an einer Stelle gebündelt. Ein `Codec` legt zwei Operationen fest. `encode` wandelt ein Python-Objekt in Bytes um, `decode` wandelt Bytes wieder in ein Python-Objekt um. Abbildung 3 zeigt die Schnittstelle des Codecs, ihre zwei Umsetzungen in der Referenzimplementierung und die Stellen, an denen sie verwendet werden. Eine Umsetzung muss dabei nur eine Bedingung erfüllen. Ein Wert muss nach dem Kodieren und anschließenden Dekodieren unverändert zurückkommen. Formal lässt sich das als

$$\text{decode} \circ \text{encode} = \text{id}_D \quad (6)$$

ausdrücken. Das Zeichen $\circ$ bezeichnet die Hintereinanderausführung, wobei die rechte Funktion zuerst angewendet wird. id_D ist die Identität auf der Menge D , also die Funktion, die jeden Wert unverändert zurückgibt. Die Formel besagt damit, dass Kodieren und anschließendes Dekodieren für jeden Wert aus D dasselbe Ergebnis liefert wie gar keine Umwandlung.

D umfasst dabei nur die Werte, die das jeweilige Format unterstützt. Für die hier eingesetzte MessagePack-Umsetzung sind das Wahrheitswerte, Ganzzahlen im Bereich von 16 Bit, Zeichenketten, Bytefolgen sowie Listen und Abbildungen über diesen Typen. Außerhalb von D gilt die Eigenschaft nicht, ein Tupel etwa kommt als Liste zurück. Die Umkehrung gilt ebenfalls nicht, da sich derselbe Wert auf mehrere Arten kodieren lässt und eine fremde Bytefolge deshalb nicht zwingend unverändert zurückkommt.

Die MessagePack-Umsetzung deckt bewusst nur den Teil des Formats ab, den das System tatsächlich austauscht. Fließkommazahlen fehlen ebenso wie die Erweiterungstypen des Formats. Übertragen werden Pixelkoordinaten, Objektreferenzen und kurze Texte, und jeder weggelassene Typ verkleinert den Decoder, der auf beiden Seiten läuft. Das ist das dritte Kriterium aus Abschnitt 5.2.6, angewandt auf die konkrete Umsetzung.

Enger als im Format vorgesehen ist auch der Wertebereich der Ganzzahlen. Er endet bei 16 Bit, obwohl MessagePack auch 32 und 64 Bit kennt. Der Grund liegt in der Laufzeitumgebung des Hubs. Sie ist ohne Unterstützung für lange Ganzzahlen übersetzt, sodass sich dort keine Konstante oberhalb von $2^{30} - 1$ darstellen lässt. Die Zweige des Encoders für größere Werte enthielten solche Konstanten und verhinderten das Laden des gesamten Moduls, unabhängig

davon, ob je ein so großer Wert übertragen wird. Für die hier auftretenden Werte reichen 16 Bit aus.

Welcher Codec verwendet wird, legt eine einzige Zuweisung fest. Möglich ist dieser Wechsel nur, weil die Rahmung die Länge der Nutzlast ausdrücklich mitführt. Die binäre Kodierung erzeugt Nullbytes, und ohne Längensfeld würde das Abschneiden von Füllbytes Teile der Nutzlast entfernen.

Zusätzlich steht ein JSON-Codec zur Verfügung. Er ist bei der Fehlersuche hilfreich, da ein mitgeschnittener Frame seine Nutzlast dann im Klartext zeigt. Wie sich die beiden Formate auf die Anzahl der Übertragungen auswirken, vergleicht Abschnitt 7.

6.5 Kommunikationsschicht

Die Kommunikationsschicht ist auf beide Seiten verteilt. Auf dem Client, also dem Hub, bildet `PyBricksInterceptor.call()` einen Methodenaufruf auf eine Nachricht ab und wartet auf das Ergebnis. Auf dem Server, also dem ESP32, nimmt `MiddlewareInterceptor` die Nachricht entgegen, übergibt sie der Verteilung und legt das Ergebnis zur Abholung bereit. Beide Klassen übernehmen zugleich Aufgaben der Transportschicht, wie Tabelle III zeigt. Die folgenden Abschnitte beschreiben nur den Teil, der zur Kommunikationsschicht gehört.

Die Umsetzung folgt dem Entwurf aus Abschnitt 5.3 und legt zwei Punkte fest, die dort nur als Prinzip beschrieben wurden. Erstens werden Argumente nach ihrer Position übertragen, wobei objektgebundene Methoden die Referenz des Objekts als erstes Argument erhalten. Zweitens werden Methoden über eine Kennung von 16 Bit angesprochen, die beide Seiten aus dem Methodennamen berechnen.

6.5.1 Aufrufnachricht

Die Nutzlast einer Nachricht tritt in drei Ausprägungen auf, je nach Richtung und Ausgang des Aufrufs.

Tabelle VI: Aufbau der Nutzlast je Ausprägung

Ausprägung	Schlüssel	Inhalt
CommandPayload	s, c, a	Wirkungsbereich, Methodenkennung, Liste der Argumente
DataPayload	d	Rückgabewert
ErrorPayload	e, k	Fehlerbeschreibung, optional die Fehlerart

Die Schlüssel sind einbuchstabig. Die Argumente stehen als Liste in der Reihenfolge der Parameter, wie sie die gemeinsamen Schnittstellen in `display/protocol` festlegen. Die Methode wird durch eine Kennung von 16 Bit bezeichnet, die Listing 5 aus ihrem Namen berechnet.

```
def name_hash(name):
    h = 5381
    for ch in name:
        h = ((h * 33) ^ ord(ch)) & 0xFFFF
    return h
```

Listing 5: Berechnung der Methodenkennung aus dem Methodennamen

Das Verfahren ist eine auf 16 Bit begrenzte Variante des bekannten djb2-Hashs. Die eingebaute Funktion `hash` ist dafür nicht verwendbar, da CPython ihr Ergebnis je Prozess zufällig verändert und MicroPython es anders berechnet. Die Maskierung nach jedem Schritt hält alle Zwischenwerte unter 2^{30} . Das ist auf dem Hub aus demselben Grund nötig wie die Begrenzung der Ganzzahlen in Abschnitt 6.4. Ein gebräuchlicher 32-Bit-Hash wie FNV würde diese Grenze überschreiten.

Die Breite von 16 Bit ist eine Abwägung. MessagePack kodiert Werte bis 65535 in drei Byte, gegenüber 13 Byte für den Namen `create_label`. Eine Kennung von acht Bit wäre ein Byte kürzer, bei zehn Methoden in einem Wirkungsbereich läge die Wahrscheinlichkeit einer Kollision dann aber bei etwa 16,3 %. Eine Kollision liegt vor, wenn zwei verschiedene Methodennamen denselben Hashwert ergeben. Die Wahrscheinlichkeit dafür steigt mit der Anzahl der Methoden pro Wirkungsbereich. Die folgende Gleichung

$$PKollision = 1 - \prod_{i=0}^{n-1} \left(1 - \frac{i}{N}\right) \quad (7)$$

schätzt die Wahrscheinlichkeit ab, dass bei n Methoden in einem Wirkungsbereich und N möglichen Kennungen mindestens zwei denselben Hashwert liefern. Sie ist nur eine Näherung, da sie gleichverteilte Hashwerte annimmt und djb2 nicht gleichverteilt ist. Nach Formel 7 liegt die Wahrscheinlichkeit einer Kollision bei 16 Bit und zehn Methoden bei etwa 0,069 %, bei 20 Methoden bei etwa 0,29 % und bei 50 Methoden bei 1,852 %.

Kollisionen sind also möglich, werden aber beim Start des Servers erkannt. Der Server trägt dazu jede Methode in ein Dictionary ein, wobei der Hashwert als Schlüssel dient. Eine zweite Methode mit demselben Hashwert führt zu einer Ausnahme, die den Start verhindert. Listing 6 zeigt die Funktion, die die Indizierung vornimmt. Sie liegt in der Basisklasse `RPCDispatcher`, von der alle Empfänger erben. Die genaue Struktur beschreibt Abschnitt 6.6.

```
def _index_methods(self):
    index = {}
    for name in dir(self):
        if name.startswith("_") or name in self.NOT_REMOTE:
            continue
        if not callable(getattr(self, name)):
            continue

        h = name_hash(name)
        if h in index:
            raise ValueError("hash collision in scope '{}': {} and {}".format(
                self.scope, index[h], name))
        index[h] = name
    return index
```

Listing 6: Funktion zur Registrierung einer Methode

Ergebnis und Fehler unterscheiden sich allein im Schlüssel. Der Client prüft, ob `e` vorhanden ist, und löst in diesem Fall eine Ausnahme aus. Welche Ausnahme das ist, bestimmt die Fehlerart unter `k`, wie Abschnitt 6.6 beschreibt. Antworten enthalten keinen Wirkungsbereich. Der Server braucht ihn zur Auswahl des Empfängers, eine Antwort geht dagegen immer an den einen Aufrufer zurück, der auf sie wartet.

6.5.2 Verteilung eingehender Aufrufe

Die Zuordnung eines Aufrufs zur ausführenden Methode erfolgt in zwei Stufen. Zunächst wird anhand des Wirkungsbereichs ein Dispatcher ausgewählt. Zur Erinnerung: Der Wirkungsbereich benennt die Art des Empfängers, der den Aufruf ausführt, und wird im `CommandPayload` unter dem Schlüssel `s` mitgeführt. Die Dispatcher sind in einem `RPCDispatcherPool` registriert, wobei jeder einen eigenen Wirkungsbereich bedient. Die Registrierung erfolgt beim Start des Servers, und der Versuch, einen Wirkungsbereich ein zweites Mal zu belegen, löst eine Ausnahme aus.

Anschließend schlägt der ausgewählte Dispatcher die Methodenkennung aus `c` nach und ruft die Methode mit den Argumenten aus `a` auf. Dass diese Kennungen innerhalb eines Wirkungsbereichs eindeutig sind, prüft der Dispatcher beim Aufbau seines Index, wie Listing 6 zeigt.

Findet sich eine Kennung nicht im Index, löst der Dispatcher eine Ausnahme aus. Sie wird in eine Fehlermeldung überführt und erreicht den Client, womit die Zusicherung aus Abschnitt 5.2.7 auch für Aufrufe gilt, die es nicht gibt.

Aufruflbar ist damit jede öffentliche Methode eines Adapters. Das ist enger als ein Zugriff über den bloßen Namen, bei dem auch Attribute wie die Registratur erreichbar wären, aber weiter gefasst als die gemeinsame Schnittstelle in `display/protocol`. Der Adapter für Screens etwa erbt die Rückrufe der Anzeigebibliothek, die ebenfalls im Index stehen. Für die betrachtete Punkt-zu-Punkt-Verbindung, in die kein Dritter Nachrichten einspeisen kann, ist das hinnehmbar. Bei mehreren Teilnehmern müsste der Index auf die Methoden der Schnittstelle beschränkt werden.

6.5.3 Rückkanal

Nach der Bestätigung des letzten Frames ist ein Aufruf zwar vollständig übertragen, aber noch nicht ausgeführt. Der Server muss das Ergebnis erst berechnen. Anschließend legt er es in einem Ausgangspuffer ab, bis der Client es abholt.

Das Abholen erfolgt über denselben anfragegetriebenen Kanal wie die Übertragung der Aufrufe. Der Client fragt in einem festen Intervall mit `READY` an, ob ein Ergebnis bereitliegt. Der Server antwortet mit `DONE`, wenn das der Fall ist, und sonst mit `NACK`. Liegt das Ergebnis vor, holt der Client es mit `NEXT` Frame für Frame ab, wobei jede Anfrage die Position des nächsten Frames angibt. Der Server beantwortet jede Anfrage mit `frame_at(index)`, sodass eine wiederholte Anfrage denselben Frame liefert.

```

if frame.opcode == OP_READY:
    if self.__is_result_ready__(frame.g_id):
        return Frame.create_done(g_id=frame.g_id).to_bytes()
    return Frame.create_no_ack(g_id=frame.g_id).to_bytes()

if frame.opcode == OP_NEXT:
    msg = self.outgoing.get(frame.g_id)
    if msg is None:
        return Frame.create_error(g_id=frame.g_id, payload=b"nomsg").to_bytes()

    res_frame = msg.frame_at(frame.frame_nr)
    if res_frame is None:
        return Frame.create_no_ack(g_id=frame.g_id, frame_nr=frame.frame_nr).to_bytes()

    return res_frame.to_bytes()

return Frame.create_error(g_id=frame.g_id, payload=b"badop").to_bytes()

```

Listing 7: Beantwortung der beiden Anfragen auf dem Server

Die Grenzen dieses Ablaufs sind als Konstanten festgelegt.

Tabelle VII: Grenzen des Rückkanals

Konstante	Wert	Bedeutung
RESULT_POLL_MS	50 ms	Abstand zwischen zwei READY-Anfragen
MAX_RESULT_WAIT_MS	5000 ms	Wartezeit, nach der ein Aufruf als gescheitert gilt
MAX_FRAME_ATTEMPTS	8	Anfragen je Position, bevor das Abholen abbricht
MAX_KEPT_RESULTS	4	Ergebnisse, die der Server zur Abholung bereithält
MAX_EVENTS	10	gepufferte Ereignisse je Objekt

Der Server behält nur die jüngsten Ergebnisse. Ältere werden verworfen, wodurch der Speicherbedarf nach K5 begrenzt bleibt. Da der Client streng nacheinander aufruft, fragt er ohnehin nur nach dem Ergebnis seines letzten Aufrufs. Ereignisse nutzen keinen eigenen Mechanismus. Der Server legt sie je Objekt in der Registratur ab und hält dort die letzten zehn vor, wobei bei Überlauf das älteste Ereignis verworfen wird. Abgefragt werden sie über einen gewöhnlichen Aufruf wie `is_button_pressed`, der das jüngste Ereignis zurückgibt. Jede Abfrage kostet damit einen vollständigen Aufruf mit Übertragung, Wartezeit und Abholung. Der Überlauf des Puffers

wird der Anwendung nicht gemeldet. Das Verhalten ist damit zwar festgelegt, wie Abschnitt 5.3 es verlangt, für die Anwendung aber nicht erkennbar.

Wie viel Zeit ein Aufruf im Warten auf das Ergebnis verbringt, misst Abschnitt 7. Reaktionen, die ohne diesen Weg auskommen, weil der Server sie selbst ausführt, behandelt Abschnitt 6.7.

6.6 Objektmodell

Das Objektmodell bildet die grafischen Elemente der Anzeige auf Objekte ab, die die Anwendung auf dem Hub wie lokale Objekte benutzt. Bildschirme, Beschriftungen und Schaltflächen heißen in der Umsetzung Screens, Labels und Buttons. Je Objekttyp gibt es eine Basisklasse in `display/protocol`. Sie wird auf dem Client von einem Stellvertreter erfüllt, der jeden Methodenaufruf weiterleitet, und auf dem Server von einem Adapter, der ihn ausführt. Die Adapter erben zusätzlich von `RPCDispatcher` und werden dadurch über ihren Wirkungsbereich erreichbar, wie Abschnitt 6.5.2 beschreibt. Die Stellvertreter erben von `RemoteObject`, das die Referenz des Objekts hält und sie jedem Aufruf voranstellt. Abbildung 4 zeigt diese Beziehungen beispielhaft.

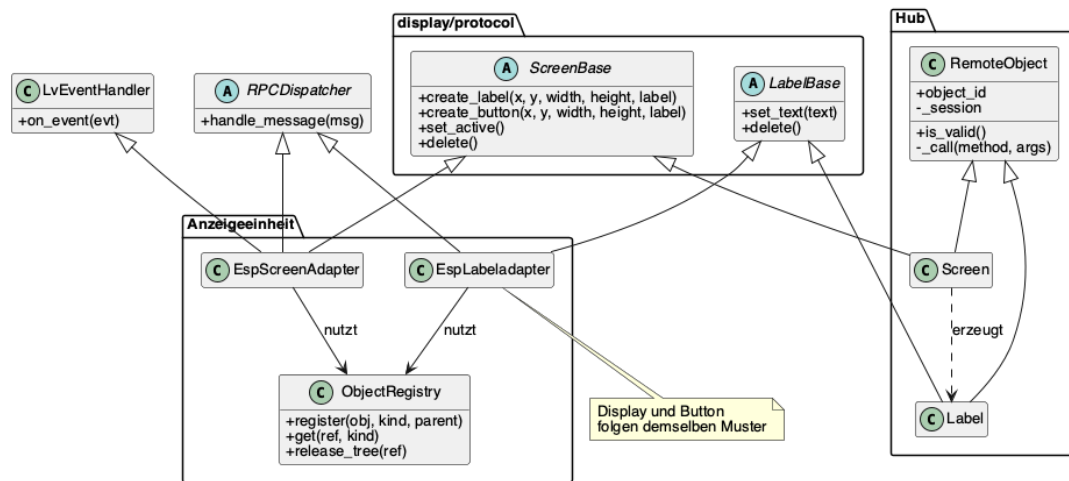


Abbildung 4: Klassen des Objektmodells auf beiden Seiten

6.6.1 Referenzen

Eine Referenz ist eine Ganzzahl von 16 Bit, die der Server beim Erzeugen eines Objekts vergibt. Sie besteht aus zwei Teilen. Die unteren zehn Bit bezeichnen einen Slot, also die Position des Objekts in der Tabelle der Registratur. Die oberen sechs Bit enthalten die Generation des Slots. Das ist ein Zähler, der bei jeder Freigabe des Slots um eins steigt. Dieses Verfahren ist als Generational Handle bekannt.

```
def _slot(self, ref):
    index = ref & SLOT_MASK
    if index >= len(self._slots):
        return None
    slot = self._slots[index]
    if slot[2] is None or slot[0] != ref >> SLOT_BITS:
        return None
    return slot
```

Listing 8: Auflösung einer Referenz in der Registratur

Ein Slot speichert neben dem Objekt und seiner Generation auch die Art des Objekts und die Referenz seines Parents. Die Art ist `screen`, `label` oder `button`. Jede Methode eines Adapters gibt beim Nachschlagen an, welche Art sie erwartet. Übergibt die Anwendung etwa die Referenz eines Screens an `set_text`, wird das erkannt, bevor auf das Objekt zugegriffen wird.

Zehn Bit erlauben 1024 gleichzeitig bestehende Objekte. Der Speicher des ESP32 ist schon bei deutlich weniger Objekten erschöpft, sodass diese Grenze in der Praxis nicht erreicht wird. Die Breite von 16 Bit folgt aus Abschnitt 6.4. Der Codec überträgt Ganzzahlen nur bis zu dieser Größe, und MessagePack kodiert eine Referenz damit in höchstens drei Byte. Eine Referenz in Textform wie `obj_12` belegt sieben Byte.

6.6.2 Lebenszyklus

Objekte entstehen immer innerhalb eines übergeordneten Objekts. Screens werden vom Display erzeugt, Labels und Buttons von einem Screen. Ein Button besteht auf dem Server aus zwei Objekten, dem Button selbst und dem Label für seine Beschriftung. Beide werden getrennt registriert, und der Stellvertreter auf dem Client erhält beide Referenzen. So lässt sich die Beschriftung über den Wirkungsbereich `label` ändern, ohne dass der Button-Adapter dafür eine eigene Methode braucht.

Die Registratur bildet dabei den Objektbaum der Anzeigebibliothek nach. Jeder Eintrag kennt seinen Parent, und `release_tree` gibt ein Objekt zusammen mit allen darin erzeugten Objekten frei. Das ist nötig, weil LVGL beim Löschen eines Objekts auch dessen Kinder löscht. Ohne den Baum würden deren Einträge in der Registratur bestehen bleiben und auf nicht mehr vorhandene Objekte verweisen.

Gelöscht wird über die Methode `delete()`, die jeder Objekttyp anbietet. Der Adapter löscht zuerst das LVGL-Objekt und gibt danach die Einträge frei. Beim Freigeben entfallen auch die Ereignisse des Objekts und alle Bindungen, an denen es beteiligt ist. Den gerade angezeigten Screen kann die Anwendung nicht löschen. LVGL legt nicht fest, was angezeigt wird, wenn der aktive Screen fehlt. Ein solcher Aufruf wird deshalb mit einem Fehler beantwortet.

`clear()` löscht alle Objekte auf einmal. Die Registratur allein zu leeren genügt dafür nicht, da LVGL die Objekte in einem eigenen Baum hält und sie nicht freigibt, wenn die letzte Python-Referenz entfällt. Der Server lädt daher zuerst einen leeren Screen und löscht anschließend jeden Screen der Registratur mit allem, was darauf liegt. Die Slots bleiben dabei erhalten und werden einzeln freigegeben, damit ihre Generationen weiterzählen.

6.6.3 Veraltete Referenzen

Nach dem Löschen eines Objekts kann dessen Stellvertreter auf dem Client weiter bestehen. Seine Referenz ist dann veraltet. Da freie Slots wiederverwendet werden, kann derselbe Slot inzwischen ein neues Objekt enthalten. Ohne Generation würde ein Aufruf über den alten Stellvertreter unbemerkt das neue Objekt verändern. Mit Generation passt die Referenz nicht mehr zum Slot, und der Aufruf wird abgewiesen.

Die Prüfung geschieht an zwei Stellen. Auf dem Server prüft die Registratur jede Referenz wie in Listing 8. Auf dem Client führt der `PyBricksInterceptor` eine Session, die bei `clear()` und bei einem Verbindungsabbruch endet. Jeder Stellvertreter merkt sich die Session, in der er entstanden ist. Gehört er zu einer früheren Session, löst er den Fehler aus, ohne eine Übertragung zu starten. Nach einem Verbindungsabbruch ist das die einzige verlässliche Prüfung, denn der ESP32 kann in der Zwischenzeit neu gestartet sein und seine Registratur von vorn aufbauen. Damit eine alte Referenz auch in diesem Fall nicht zufällig passt, beginnen die Generationen nach einem Start bei einem zufälligen Wert.

Beide Prüfungen melden denselben Fehler. Die Fehlernutzlast trägt dafür neben der Beschreibung unter `e` eine Fehlerart unter `k`, und der Client löst anhand dieser Art die passende Ausnahme aus. `StaleReferenceError` steht für ein nicht mehr vorhandenes Objekt, `WrongKindError` für eine Referenz der falschen Art. Alle übrigen Fehler kommen als `RemoteError` an. Alle drei erben von `RuntimeError`. Die Anwendung kann dadurch gezielt auf eine veraltete Referenz reagieren und beispielsweise die Oberfläche neu aufbauen, ohne den Fehlertext auswerten zu müssen.

Ganz ausgeschlossen sind Verwechslungen allerdings nicht. Nach 64 Freigaben desselben Slots wiederholt sich die Generation, und ein Stellvertreter, der die ganze Zeit bestanden hat,

passt wieder. Löscht die Anwendung einen Screen, bleiben außerdem die Stellvertreter der Elemente darauf auf dem Client gültig, da der Client sie nicht kennt. Ihr nächster Aufruf erreicht den Server und wird erst dort abgewiesen.

6.7 Ansteuerung der Anzeige

Die Ansteuerung der Anzeige bildet auf dem Server die oberste Schicht aus Abschnitt 4. Sie ist die einzige Stelle mit Kenntnis der Anzeigehardware und liegt in `display/server/driver`. Die Darstellung übernimmt **LVGL** in der MicroPython-Anbindung, die auch den Treiber für den Displaycontroller ILI9341 mitbringt. Das Display ist über **SPI** angebunden, der kapazitive Touchcontroller über **I2C**.

Aufgebaut wird die Anzeige vom `ESPDisplayAdapter`. Er öffnet die **I2C**-Verbindung, erzeugt den Touchcontroller und übergibt ihn dem Displaytreiber, der ihn als Zeigegerät bei **LVGL** anmeldet. Anschließend reicht er das **LVGL**-Modul an die Adapter für Screens, Labels und Buttons weiter. Nur diese Adapter rufen **LVGL** auf. Die Schichten darunter kennen die Bibliothek nicht.

Das Zeichnen ist von der Ausführung der Aufrufe getrennt. Der Treiber startet beim Initialisieren eine Ereignisschleife, die in festem Takt über einen Hardware-Timer den Task-Handler von **LVGL** einplant. Dieser zeichnet geänderte Bereiche neu und fragt die Eingabegeräte ab. Ein Adapter ändert also nur den Zustand eines Objekts, etwa den Text eines Labels. Sichtbar wird die Änderung erst beim nächsten Durchlauf des Task-Handlers. Ein Aufruf kann deshalb bereits abgeschlossen sein, bevor die Änderung auf dem Display erscheint.

Der Touchcontroller wird regelmäßig abgefragt, seine Interrupt-Leitung wird nicht genutzt. **LVGL** ruft dazu bei jedem Durchlauf die Funktion `touch_read_cb` auf, die das Registerabbild des Controllers liest. Jede Abfrage kostet eine **I2C**-Transaktion, auch wenn keine Berührung vorliegt. Die Funktion darf außerdem keine Ausnahme weitergeben. Bei einer unbehandelten Ausnahme beendet sich die Ereignisschleife dauerhaft. Die Anzeige würde danach nicht mehr neu gezeichnet, obwohl weiterhin Aufrufe ankommen. Ein fehlgeschlagener Lesevorgang wird deshalb als fehlende Berührung gemeldet.

Erkennt **LVGL** eine Berührung auf einem Button, löst es ein Ereignis aus. Der Screen-Adapter registriert dafür beim Erzeugen eines Buttons die Methode `on_event` für die Ereignisse `PRESSED` und `RELEASED`. `on_event` ermittelt über die Registratur die Referenz des Buttons, übersetzt den Ereigniscode in einen Namen wie `press` und ergänzt einen Zeitstempel. Das Ergebnis wird im Ereignispuffer des Objekts abgelegt, aus dem der Client es wie in Abschnitt 6.5 beschrieben abfragt.

Vor dem Ablegen prüft `on_event`, ob für das Ereignis eine Bindung besteht. Eine Bindung legt fest, dass ein Ereignis auf einem Objekt eine Aktion auf einem anderen auslöst. Der Client legt sie einmal mit einem gewöhnlichen Aufruf an. Danach führt der Server die Aktion direkt im Rückruf aus, ohne dass eine Übertragung nötig ist. Der Client erfährt trotzdem von jedem Ereignis, da es unabhängig von der Bindung gepuffert wird.

Tabelle VIII: Aktionen, die an ein Ereignis gebunden werden können

Wert	Name	Wirkung auf das Ziel
0x1	ACT_SHOW_SCREEN	Screen laden und anzeigen
0x2	ACT_SHOW	Objekt sichtbar machen
0x3	ACT_HIDE	Objekt verbergen
0x4	ACT_TOGGLE	Sichtbarkeit umkehren

Die Menge der Aktionen ist absichtlich klein und fest vorgegeben. Sie enthält keine Bedingungen, keinen Zustand und keine Berechnungen. Bindungen bleiben damit auf Reaktionen beschränkt, die nur die Darstellung betreffen, etwa den Wechsel zwischen zwei Screens. Die Logik der

Anwendung liegt weiterhin auf dem Client. Unbekannte Aktionen weist `bind` ebenso zurück wie Referenzen, die veraltet sind oder die falsche Art haben (Abschnitt 6.6).

6.8 Ablauf eines vollständigen Aufrufs

6.9 Nebenläufigkeit und Fehlerbehandlung

7 Evaluation

7.1 Funktionaler Nachweis

7.2 Übertragungsaufwand

7.3 Ressourcenverbrauch

7.4 Bewertung des Hardwareaufbaus

7.5 Diskussion der Ergebnisse

8 Fazit und Ausblick

8.1 Limitierungen

8.1.1 Kommunikationsmodell

8.1.2 Latenz und Durchsatz

8.1.3 Abstraktionsgrenzen

8.2 Übertragbarkeit auf andere Verbindungsklassen

8.3 Mögliche Erweiterungen

8.3.1 Ereignisgetriebener Rückkanal

8.3.2 Alternative Serialisierung

8.3.3 Integrierter Hardwareaufbau

8.3.4 Weitere Peripherieklassen

8.4 Zusammenfassung

9 Abbildungs- und Tabellenverzeichnis

Abbildungsverzeichnis

Abbildung 1	Schichtenmodell der Architektur. Beide Seiten sind gleich aufgebaut und verkehren jeweils nur mit der unmittelbar darunterliegenden Schicht.	15
Abbildung 2	Ablauf eines Aufrufs an der Interceptor-Schnittstelle. Jede der drei Phasen besteht aus Wiederholungen derselben Operation, und jede wird vom Client angestoßen.	20
Abbildung 3	Klassendiagramm des Codecs	32
Abbildung 4	Klassen des Objektmodells auf beiden Seiten	36

Tabellenverzeichnis

Tabelle I	Vergleich der betrachteten Ansätze entlang der Auswahlkriterien	8
Tabelle II	Gegenüberstellung der Kommunikationsanforderungen und der Einschränkungen der Plattform	13
Tabelle III	Zuordnung der Module zu den Schichten aus Abschnitt 4	28
Tabelle IV	Aufbau des Frame-Headers	29
Tabelle V	Werte für die Art des Frames und ihre Bedeutung	30
Tabelle VI	Aufbau der Nutzlast je Ausprägung	33
Tabelle VII	Grenzen des Rückkanals	35
Tabelle VIII	Aktionen, die an ein Ereignis gebunden werden können	38

Code-Listings

Listing 1	Registrierung des einzigen Kommandos	28
Listing 2	Funktion zur Erzeugung eines Frames aus Bytes	30
Listing 3	Funktion zur Umwandlung eines Frames in Bytes	30
Listing 4	Funktion zur Erzeugung von Frames aus einer Nachricht	31
Listing 5	Berechnung der Methodenkennung aus dem Methodennamen	33
Listing 6	Funktion zur Registrierung einer Methode	34
Listing 7	Beantwortung der beiden Anfragen auf dem Server	35
Listing 8	Auflösung einer Referenz in der Registratur	36

10 Abkürzungsverzeichnis

ARQ – Automatic Repeat reQuest
CBOR – Concise Binary Object Representation
CORBA – Common Object Request Broker Architecture
gRPC – gRPC Remote Procedure Call
HTTP – Hypertext Transfer Protocol
I2C – Inter-Integrated Circuit
JSON – JavaScript Object Notation
LED – Light Emitting Diode
LPF2 – LEGO Power Functions 2.0
LVGL – Light and Versatile Graphics Library
MQTT – Message Queuing Telemetry Transport
MTU – Maximum Transmission Unit
ONC RPC – Open Network Computing Remote Procedure Call
OSI – Open Systems Interconnection
PUP – LEGO Powered Up
REST – Representational State Transfer
RFC – Request for Comments
RMI – Remote Method Invocation
RPC – Remote Procedure Call
RTU – Remote Terminal Unit
SPI – Serial Peripheral Interface
SRAM – Static Random Access Memory
TCP – Transmission Control Protocol
TFT – Thin-Film-Transistor
UART – Universal Asynchronous Receiver Transmitter
UDP – User Datagram Protocol
XML – Extensible Markup Language

Bibliografie

- [1] A. D. Birrell und B. J. Nelson, „Implementing Remote Procedure Calls“, *ACM Transactions on Computer Systems*, Bd. 2, Nr. 1, S. 39–59, Feb. 1984, doi: [10.1145/2080.357392](https://doi.org/10.1145/2080.357392).
- [2] J. Waldo, G. Wyant, A. Wollrath, und S. Kendall, „A Note on Distributed Computing“, Technical Report TR-94-29, Nov. 1994. [Online]. Verfügbar unter: <https://scholar.harvard.edu/waldo/publications/note-distributed-computing>
- [3] A. S. Tanenbaum und M. Van Steen, *Distributed Systems*, 3. Aufl. Maarten van Steen, 2017. [Online]. Verfügbar unter: <https://www.distributed-systems.net/index.php/books/ds3/>
- [4] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, und M. Stal, *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*. Chichester: John Wiley & Sons, 1996.
- [5] E. Gamma, R. Helm, R. Johnson, und J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.
- [6] „Common Object Request Broker Architecture (CORBA) Specification, Version 3.4“. Zugegriffen: 14. September 2026. [Online]. Verfügbar unter: <https://www.omg.org/spec/CORBA/>
- [7] „Java Remote Method Invocation Specification“. Zugegriffen: 14. September 2026. [Online]. Verfügbar unter: <https://docs.oracle.com/javase/8/docs/platform/rmi/spec/rmiTOC.html>
- [8] C. Bormann, M. Ersue, und A. Keränen, „Terminology for Constrained-Node Networks“, Request for Comments RFC7228, Mai 2014. doi: [10.17487/RFC7228](https://doi.org/10.17487/RFC7228).
- [9] „MicroPython - Python for Microcontrollers“. Zugegriffen: 14. September 2026. [Online]. Verfügbar unter: <https://micropython.org/>
- [10] „I2C Bus“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://de.i2c-bus.org/>
- [11] „Mct.de: SPI - Serial Peripheral Interface“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://web.archive.org/web/20190116220910/http://www.mct.de/faq/spi.html>
- [12] „MODBUS Application Protocol Specification V1.1b3“. Zugegriffen: 14. September 2026. [Online]. Verfügbar unter: https://modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf
- [13] „CICS Transaction Server for z/OS“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://www.ibm.com/docs/en/cics-ts/5.5.0?topic=rpc-onc-concepts>
- [14] S. Microsystems, „RPC: Remote Procedure Call Protocol Specification“, Request for Comments RFC1050, Apr. 1988. doi: [10.17487/RFC1050](https://doi.org/10.17487/RFC1050).
- [15] R. Thurlow, „RPC: Remote Procedure Call Protocol Specification Version 2“, Request for Comments RFC5531, Mai 2009. doi: [10.17487/RFC5531](https://doi.org/10.17487/RFC5531).
- [16] „JSON-RPC 2.0 Specification“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://www.jsonrpc.org/specification>
- [17] „gRPC“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://grpc.io/>
- [18] T. Bray, „The JavaScript Object Notation (JSON) Data Interchange Format“, Request for Comments RFC8259, Dez. 2017. doi: [10.17487/RFC8259](https://doi.org/10.17487/RFC8259).
- [19] „MessagePack: It's like JSON. but Fast and Small.“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://msgpack.org/>
- [20] C. Bormann und P. E. Hoffman, „Concise Binary Object Representation (CBOR)“, Request for Comments RFC8949, Dez. 2020. doi: [10.17487/RFC8949](https://doi.org/10.17487/RFC8949).
- [21] „CBOR — Concise Binary Object Representation | Overview“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://cbor.io/>

- [22] „Protocol Buffers“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://protobuf.dev/>
- [23] „Nanopb — Zephyr Project Documentation“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://docs.zephyrproject.org/latest/services/serialization/nanopb.html>
- [24] „Firmata/Arduino“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://github.com/firmata/arduino>
- [25] „MQTT - The Standard for IoT Messaging“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://mqtt.org/>
- [26] „LEGO UART Sensor Protocol“. Zugegriffen: 14. September 2026. [Online]. Verfügbar unter: <https://github.com/pybricks/technical-info/blob/master/uart-protocol.md>
- [27] „PUPRemote Documentation — Anton's Mindstorms Docs Documentation“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://docs.antonsmindstorms.com/en/latest/Software/PUPRemote/docs/index.html>
- [28] „Handlungsorientiertes Lernen“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://education.lego.com/de-de/>
- [29] „STM32F413/423 - STMicroelectronics“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://www.st.com/en/microcontrollers-microprocessors/stm32f413-423.html>
- [30] „Lego::Lab Karlsruhe“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://legolab-karlsruhe.de/mindstorms/spikeprime>
- [31] „LVGL — Light and Versatile Embedded UI Ecosystem“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://lvgl.io/>
- [32] „LMS-ESP32 v2.0: The Clever Wi-Fi and Gamepad Board for LEGO SPIKE“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://www.antonsmindstorms.com/product/powerful-lms-esp32-board-for-spike-and-mindstorms/>
- [33] „Pybricks“. Zugegriffen: 15. September 2026. [Online]. Verfügbar unter: <https://pybricks.com/>
- [34] A. S. Tanenbaum und D. Wetherall, *Computernetzwerke*, 5., aktualisierte Auflage. in Always learning. München [u.a.]: Pearson, 2012.
- [35] „How Encapsulation Works Within the TCP/IP Model : Learn-Networking.Com“. Zugegriffen: 1. September 2026. [Online]. Verfügbar unter: <https://web.archive.org/web/20120807024653/http://learn-networking.com/tcp-ip/how-encapsulation-works-within-the-tcpip-model>
- [36] „Architectural Styles and the Design of Network-based Software Architectures“. Zugegriffen: 9. September 2026. [Online]. Verfügbar unter: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>